一、什么是数据结构

1、数据结构的起源

- 数据结构不是研究数值计算的这些是数学家应该研究的问题，它是研究计算机存储、组织数据的方式问题的学科，数据结构会影响算法的效率，合适的数据结构可以带来更高的运行或存储效率。
- 1968年，美国的高纳德（Donald E. Knuth）教授《基本算法》，开创了数据结构课程体系的先河。
- 程序设计 = 数据结构 + 算法
    - 凭借一句话获得图灵奖的Pascal之父——Nicklaus Wirth，让他获得图灵奖的这句话就是他提出的著名公式：“算法+数据结构=程序”。
    - 这个公式对计算机科学的影响程度足以类似物理学中爱因斯坦的“E=MC^2”——一个公式展示出了程序的本质。

2、数据结构的基本概念

- 数据(data)：所有能输入到计算机中去的描述客观事物的符号
- 数据项(data item)：有独立含义的数据最小单位，也称域(field)
- 数据元素(data element)：数据结构的基本单位，也称节点（node）或记录（record）
- 数据结构(data structure)：数据元素和数据元素关系的集合
- 算法(algorithm)：是对特定问题求解步骤的一种描述，是指令的有限序列

```c
#include <stdio.h>

typedef struct Student
{
    int id;         // 结构成员就是 数据项
    char name[20];
    char sex;
    short age;
    float score;
}Student;

int main(int argc,const char* argv[])
{
    Student stu = { // 结构变量 stu 是数据元素，负责给他初始化的叫数据
        10086,
        "hehe",
        'w',
        23,
        88.5
    };

    // 数据结构
    Student stus[5] = {
        {10010,"hehe1",'m',18,92},
        {10011,"hehe2",'w',18,91},
        {10012,"hehe3",'m',17,93},
        {10013,"hehe4",'w',18,96},
        {10014,"hehe5",'m',15,95},
    };

```

```c
    // 算法
    for(int i=0; i<5; i++)
    {
        printf("%d %s %c %hd %g\n",
            stus[i].id,
            stus[i].name,
            stus[i].sex,
            stus[i].age,
            stus[i].score);
    }
}
```

3、数据结构的三个方面

- 数据的逻辑结构：元素之间具有什么关系。
- 数据的存储结构：元素在内存中如何存储的。
- 数据结构的运算：数据结构应具有的功能或擅长的功能。

4、数据元素之间的逻辑结构有四种基本类型

- 集合：数据元素之间除了同属一个集合外没有其它关系。
- 线性结构：数据元素之间存在一对一的关系，也被称为线性表，简称为表，最具代表性的就是数组、链表。
- 树型结构：数据元素之间存在一对多的关系，如：族谱关系、组织关系。
- 图状结构：数据元素之间存在多对多的关系，如：地铁、高铁线路图。

二、数据结构的存储方式

数据结构在计算机内存中的存储包括数据元素的存储和元素关系的表示。

元素之间的关系在计算机中有两种不同的表示方法，顺序表示和非顺序表示。

1、顺序存储结构

数据存储在一段连续的空间中，用数据元素在存储器中的相对位置来表示数据元素的关系。

- 优点：随机访问方便，访问效率极高。
- 缺点：插入删除不方便。

2、链式存储结构

结构中的数据元素存放在彼此独立的地址空间中，每个独立的地址空间称为节点。

在每一个数据元素中增加一个存放另一个元素地址的指针，用来表示数据元素之间的关系。

- 优点：插入删除方便，空间利用率高。
- 缺点：随机访问不方便，只能由前到后逐个访问。

3、逻辑结构和存储结构的关系

数据结构	表	树	图
顺序	顺序表	顺序树	矩阵
链式	链式表	链式树	邻接表(顺序+链式)

每种逻辑结构采用何种物理结构实现，并没有明确规定，通常根据实现的难易程度，以及在时间和空间方面的要求，选择最适合的物理结构，也不排除复合多种物理结构实现一种逻辑结构的可能。

三、线性表

1、线性结构

在数据元素的非空有限集中：

- 存在唯一的一个被称做“第一个”的数据元素
- 存在唯一的一个被称做“最后一个”的数据元素
- 除第一个之外，集合中的每个数据元素均只有一个前驱
- 除最后一个之外，集合中的每个数据元素均只有一个后继

2、线性表

线性表是n个数据元素的有限序列，同一线性表中的元素必定具有相同特性，相伶的数据元素之间存在着序偶关系。

线性表中元素的个数n(n>=0)定义为线性表的长度，0==n时称为空表，在非空表中每个数据元素都有一个确定的位置(下标)。

线性表是一个相当灵活的数据结构，它的长度可根据需要增长或缩短。

3、线性表的顺序表示和实现

线性表的顺序表示指的是用一组地址连续的存储单元依次存储线性表的数据元素。

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#define TYPE int

// 设计数据结构
typedef struct Array
{
    TYPE* base; // 存储元素的基址
    size_t cnt; // 元素的数量
    size_t cap; // 表的容量
}Array;

// 创建线性表,调用者需要提供容量参数，成功返回线性表指针
Array* creat_array(size_t cap)
{
    // 为线性表创建内存空间
    Array* array = malloc(sizeof(Array));
    // 创建存储元素的内存空间
    array->base = malloc(sizeof(TYPE)*cap);
    // 初始化数量成员
    array->cnt = 0;
    // 备份线性表的容量
    array->cap = cap;
    // 返回线性表指针
    return array;
}

// 销毁线性表,调用者只需要提供线性表指针即可
```

```c
void destory_array(Array* array)
{
    // 释放存储元素的内存空间
    free(array->base);
    // 释放线性表内存空间
    free(array);
}

// 清理所有元素
void clear_array(Array* array)
{
    array->cnt = 0;
}

// 判断线性表是否是空表
bool empty_array(Array* array)
{
    return 0 == array->cnt;
}

// 求线性表的长度
size_t length_array(Array* array)
{
    return array->cnt;
}

// 访问指定位置的元素
bool get_array(Array* array,int index,TYPE* elemp)
{
    if(index >= array->cnt)
        return false;

    *elemp = array->base[index];
    return true;
}

// 在末尾添加元素
void add_back_array(Array* array,TYPE elem)
{
    // 容量如果不够用
    if(array->cnt >= array->cap)
    {
        // 容量扩展两倍
        array->cap *= 2;
        // 元素存储空间扩展两倍
        array->base = realloc(array->base,sizeof(TYPE)*array->cap);
    }

    // 末尾添加元素
    array->base[array->cnt++] = elem;
}

// 插入元素
bool insert_array(Array* array,int index,TYPE elem)
{
    // 如果下标不合法则插入失败
    if(index >= array->cnt)
        return false;
```

```

    // 把最后一个元素添加到末尾
    add_back_array(array,array->base[array->cnt-1]);

    // 线性表的顺序存储才可以使用
    memmove(array->base+index+1,array->base+index,sizeof(TYPE)*(array->cnt-index-2));

    array->base[index] = elem;
    return true;
}

// 删除元素，按位置删除
bool delete_index_array(Array* array,int index)
{
    if(index >= array->cnt)
        return false;

    memmove(array->base+index,array->base+index+1,sizeof(TYPE)*(array->cnt-index-1));
    array->cnt--;
    return true;
}

// 查询元素
int query_array(Array* array,TYPE elem,int (*compare)(const void*,const void*))
{
    for(int i=array->cnt-1; i>=0; i--)
    {
        if(!compare(&elem,array->base+i))
            return i;
    }
    return -1;
}

// 删除元素，按值删除
bool delete_value_array(Array* array,TYPE elem,int (*compare)(const void*,const void*))
{
    return delete_index_array(array,query_array(array,elem,compare));
}

// 对线性表进行排序
void sort_array(Array* array,int (*compare)(const void*,const void*))
{
    bool flag = true;
    for(int i=array->cnt-1; flag && i>0; i--)
    {
        flag = false;
        for(int j=0; j<i; j++)
        {
            if(1 == compare(array->base+j,array->base+j+1))
            {
                TYPE tmp = array->base[j];
                array->base[j] = array->base[j+1];
                array->base[j+1] = tmp;
                flag = true;
```

```
            }
        }
    }
}

// 遍历线性表,只是为了测试
void show_array(Array* array)
{
    for(int i=0; i<array->cnt; i++)
    {
        printf("%d ",array->base[i]);
    }
    printf("\n");
}

int main(int argc,const char* argv[])
{
    Array* array = creat_array(10);
    for(int i=0; i<10; i++)
    {
        add_back_array(array,rand()%100);
    }

    show_array(array);

    int intcmp(const void* p1,const void* p2)
    {
        if(*(int*)p1 > *(int*)p2)
            return 1;
        if(*(int*)p1 < *(int*)p2)
            return -1;
        return 0;

    }

    delete_value_array(array,77,intcmp);

    sort_array(array,intcmp);

    show_array(array);
    destory_array(array);
    return 0;
}
```

作业：把顺序线性表的成员指针实现方式改为柔性数组。

```
// 设计数据结构
typedef struct Array
{
    size_t cnt;     // 元素的数量
    size_t cap;     // 表的容量
    TYPE base[];    // 存储元素的柔性数组
}Array;

// 创建线性表,调用者需要提供容量参数，成功返回线性表指针
Array* creat_array(size_t cap)
{
```

```c
    // 为线性表创建内存空间
    Array* array = malloc(sizeof(Array)+sizeof(TYPE)*cap);
    // 初始化数量成员
    array->cnt = 0;
    // 备份线性表的容量
    array->cap = cap;
    // 返回线性表指针
    return array;
}

// 销毁线性表,调用者只需要提供线性表指针即可
void destory_array(Array* array)
{
    // 释放线性表内存空间
    free(array);
}

// 在末尾添加元素
void add_back_array(Array* array,TYPE elem)
{
    // 容量如果不够用
    if(array->cnt >= array->cap)
    {
        // 容量扩展两倍
        array->cap *= 2;
        // 元素存储空间扩展两倍
        array = realloc(array,sizeof(Array)+sizeof(TYPE)*array->cap);
    }

    // 末尾添加元素
    array->base[array->cnt++] = elem;
}
```

4、线性表的链式表示和实现

线性表的顺序存储结构的特点是逻辑关系上相邻的两个元素在物理位置上也相邻，因此可以随机存取表中的任一元素，它的存储位置可以用一个简单、直观的公式来表示。

这个特点也铸成了这种存储结构的缺点：在插入、删除操作时，需要移动大量的元素。而线性表的另一种表示方法——链存储结构刚好弥补了它的缺点。

链存储结构不要求逻辑上相邻的元素在物理位置上也相邻，因此它没有顺序存储结构所有具有的缺点，但同时也失去了顺序存储结构可随机存取的优点。

链存储结构的特点是元素可以使用存储内存中的任何位置（可以是连续的，也可以不连续），元素a[i]和a[i+1]的逻辑关系不依靠相对位置，而是元素中增加一个指示其后继元素的数据（元素指针），元素本身的数据+后继信息构成了存储映像，俗称节点(node)。

```c
typedef struct Node
{
    TYPE data;  // 数据域
    struct Node* next;  // 指针域
}Node;
```

若干个元素节点通过指针域连接起来，形成的线性表结构称为链式表，简称链表，节点中只有一个指向后继元素的指针域，这种链表也被称为单向链表。

单向链表必须有一个指向第一个节点的指针，被称为头指针，被它指向的节点也被称为头节点，头节点可以不存储数据，单纯的作为一个占位节点，最后一个节点指向空，作为结束标志。

1、不带头节点的单向链表

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define TYPE int

typedef struct Node
{
    TYPE data;
    struct Node* next;
}Node;

// 创建节点
Node* create_node(TYPE data)
{
    // 创建节点内存
    Node* node = malloc(sizeof(Node));
    // 赋值数据域
    node->data = data;
    // 初始化指针域
    node->next = NULL;
    return node;
}

// 头添加元素
void front_list(Node** head,TYPE data)
{
    // 创建节点
    Node* node = create_node(data);
    node->next = *head;
    *head = node;
}

// 删除元素
bool delete_index_list(Node** head,int index)
{
    // 删除每个节点，因为第一个节点没有前驱
    if(0 == index)
    {
        Node* node = *head;
        *head = (*head)->next;
        free(node);
        return true;
    }

    // 找到要删除的节点的前驱
    Node* prev = *head;
    while(NULL!=prev->next && index-->1)
            prev = prev->next;

    if(NULL != prev->next)
    {
        // 备份要删除的节点
```

```
        Node* node = prev->next;
        // 前驱节点的指针域指向后继节点
        prev->next = prev->next->next;
        free(node);
        return true;
    }

    return false;
}

// 插入元素
bool insert_list(Node** head,int index,TYPE data)
{
    Node* node = create_node(data);
    if(0 == index)
    {
        node->next = *head;
        *head = node;
        return true;
    }

    Node* prev = *head;
    while(NULL!=prev->next && index-->1)
            prev = prev->next;

    if(NULL != prev->next)
    {
        node->next = prev->next;
        prev->next = node;
        return true;
    }

    return false;
}

// 遍历链表
void show_list(Node* head)
{
    for(Node* n=head; NULL!=n; n=n->next)
    {
        printf("%d ",n->data);
    }
    printf("\n");
}

int main(int argc,const char* argv[])
{
    // 创建头指针
    Node* head = NULL;
    for(int i=0; i<10; i++)
    {
        front_list(&head,i);
    }
    show_list(head);
    delete_index_list(&head,9);
    insert_list(&head,9,100);
    show_list(head);
    return 0;
```

```
}
```

2、带头节点的单向链表

在执行链表的插入、删除操作时，需要被操作节点的前驱节点和后继节点，如果被操作节点是头节点，则它没有前驱节点，需要额外特殊处理，因此为了方便插入和删除操作所以给单链表增加一个空的头节点。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define TYPE int

typedef struct Node
{
    TYPE data;
    struct Node* next;
}Node;

// 创建节点
Node* create_node(TYPE data)
{
    Node* node = malloc(sizeof(Node));
    node->data = data;
    node->next = NULL;
    return node;
}


static bool _insert_list(Node* prev,TYPE data)
{
    if(NULL == prev)
        return false;

    Node* node = create_node(data);
    node->next = prev->next;
    prev->next = node;
    return true;
}

// 根据前驱节点删除节点
static bool _delete_list(Node* prev)
{
    if(NULL == prev || NULL == prev->next)
        return false;

    Node* node = prev->next;
    prev->next = node->next;
    free(node);
    return true;
}

// 返回index下标的前驱节点
static Node* _index_list(Node* head,int index)
{
    // 找下标为i的节点的前驱
```

```
    Node* prev = head;
    while(NULL != prev->next && index-- >= 1)
        prev = prev->next;

    // index 非法，超出了节点的数量
    if(NULL == prev->next || index < -1)
        return NULL;

    return prev;
}

// 查询出值为data的前驱节点
static Node* _query_list(Node* head,TYPE data)
{
    Node* prev = head;
    while(NULL != prev->next && data != prev->next->data)
        prev = prev->next;

    if(NULL == prev->next)
        return NULL;

    return prev;
}

// 创建带头的空链表
Node* create_list(void)
{
    Node* node = malloc(sizeof(Node));
    node->next = NULL;
    return node;
}

// 销毁链表
void destory_list(Node* head)
{
    while(NULL != head)
    {
        Node* node = head;
        head = head->next;
        free(node);
    }
}

// 在头部添加元素
void front_list(Node* head,TYPE data)
{
    _insert_list(head,data);
}

// 在指定位置插入元素
bool insert_list(Node* head,int index,TYPE data)
{
    return _insert_list(_index_list(head,index),data);
}

// 删除指定位置的元素
bool delete_index_list(Node* head,int index)
{
```

```
    return _delete_list(_index_list(head,index));
}

// 按值删除元素
bool delete_value_list(Node* head,TYPE data)
{
    return _delete_list(_query_list(head,data));
}

// 按值修改
bool modify_value_list(Node* head,TYPE old,TYPE new)
{
    Node* prev = _query_list(head,old);
    if(NULL == prev)
        return false;

    prev->next->data = new;
    return true;
}

// 按位置修改
bool modify_index_list(Node* head,int index,TYPE new)
{
    Node* prev = _index_list(head,index);
    if(NULL == prev)
        return false;

    prev->next->data = new;
    return true;
}

// 访问指定位置的元素
bool get_list(Node* head,int index,TYPE* data)
{
    Node* prev = _index_list(head,index);
    if(NULL == prev)
        return false;

    *data = prev->next->data;
    return true;
}

// 查询元素
int query_list(Node* head,TYPE key)
{
    int index = 0;
    for(Node* n=head->next; NULL!=n; n=n->next)
    {
        if(n->data == key)
            return index;
        index++;
    }

    return -1;
}

// 经典排序
void sort_list(Node* head)
```

```
{
    for(Node* i=head->next; NULL!=i->next; i=i->next)
    {
        for(Node* j=i->next; NULL!=j; j=j->next)
        {
            if(i->data > j->data)
            {
                TYPE tmp = i->data;
                i->data = j->data;
                j->data = tmp;
            }
        }
    }
}

void show_list(Node* head)
{
    // 要跳过头节点
    for(Node* n=head->next; NULL!=n; n=n->next)
    {
        printf("%d ",n->data);
    }
    printf("\n");
}

int main(int argc,const char* argv[])
{
    Node* list = create_list();
    for(int i=0; i<10; i++)
    {
        front_list(list,rand()%100);
    }
    show_list(list);
    //delete_index_list(list,-1);
    //delete_value_list(list,9);
    insert_list(list,9,100);
    sort_list(list);
    show_list(list);
    return 0;
}
```

5、双向链表

所谓的双向链表就是链表节点中有两个指针域，一个指向前一个节点，叫前驱指针一般取名为prev，另一个指向后一个节点，叫后继指针，一般取名为next，双向链表的优点是可以从后向前遍历链表，对于链表后半部分的节点的访问效率大大提升。

根据之前的经验，当从前往后操作链表时，有一个空的头节点，方便操作，但现在我们需要从后往前操作链表，所以也需要一个空的尾节点。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define TYPE int

// 双向链表节点
```

```c
typedef struct Node
{
    struct Node* prev;  // 前驱指针域
    TYPE data;          // 数据域
    struct Node* next;  // 后继指针域
}Node;

// 创建节点
Node* create_node(TYPE data)
{
    Node* node = malloc(sizeof(Node));
    node->data = data;
    node->prev = NULL;
    node->next = NULL;
    return node;
}

typedef struct DoubleList
{
    Node* head;
    Node* tail;
    size_t size;
}DoubleList;

void _insert_list(Node* prev,Node* next,TYPE data)
{
    Node* node = create_node(data);
    node->next = next;
    node->prev = prev;
    prev->next = node;
    next->prev = node;
}

// 创建链表
DoubleList* create_list(void)
{
    DoubleList* list = malloc(sizeof(DoubleList));
    list->head = malloc(sizeof(Node));
    list->tail = malloc(sizeof(Node));
    list->head->prev = list->tail->next = NULL;
    list->head->next = list->tail;
    list->tail->prev = list->head;
    list->size = 0;
    return list;
}

// 销毁链表
void destroy_list(DoubleList* list)
{
    Node* node = list->tail;
    while(NULL!=node)
    {
        Node* temp = node;
        node = node->prev;
        free(temp);
    }
}

```

```c
// 头添加
void add_head_list(DoubleList* list,TYPE data)
{
    _insert_list(list->head,list->head->next,data);
    list->size++;
}

// 尾添加
void add_tail_list(DoubleList* list,TYPE data)
{
    _insert_list(list->tail->prev,list->tail,data);
    list->size++;
}

// 遍历
void show_list(DoubleList* list)
{
    for(Node* n=list->head->next; n!=list->tail; n=n->next)
    {
        printf("%d ",n->data);
    }
    printf("\n");
    for(Node* n=list->tail->prev; n!=list->head; n=n->prev)
    {
        printf("%d ",n->data);
    }
    printf("\n");
}

// 判断链表是否是空表
bool empty_list(DoubleList* list)
{
    return 0 == list->size;
}

void _del_list(Node* node)
{
    Node* prev = node->prev;
    Node* next = node->next;
    prev->next = next;
    next->prev = prev;
    free(node);
}

// 头删除
bool del_head_list(DoubleList* list)
{
    if(empty_list(list))
        return false;

    _del_list(list->head->next);
    list->size--;
    return true;
}

// 尾删除
bool del_tail_list(DoubleList* list)
{
```

```
    if(empty_list(list))
        return false;

    _del_list(list->tail->prev);
    list->size--;
    return true;
}

Node* _index_list(DoubleList* list,int index)
{
    if(index >= list->size)
        return NULL;

    if(index < list->size/2)
    {
        Node* node = list->head->next;
        while(index--)
            node = node->next;
        return node;
    }
    else
    {
        Node* node = list->tail->prev;
        while(list->size > ++index)
            node = node->prev;
        return node;
    }
}

// 指定位置插入
bool insert_index_list(DoubleList* list,int index,TYPE data)
{
    Node* node = _index_list(list,index);
    if(NULL == node)
        return false;

    _insert_list(node->prev,node,data);
    list->size++;
    return true;
}

// 指定位置删除
bool del_index_list(DoubleList* list,int index)
{
    Node* node = _index_list(list,index);
    if(NULL == node)
        return false;

    _del_list(node);
    list->size--;
    return true;

}

// 查询
Node* query_list(DoubleList* list,TYPE data)
{
    Node* left = list->head , *right = list->tail;
```

```
    do{
        left = left->next;
        right = right->prev;
        if(data == left->data)
            return left;
        if(data == right->data)
            return right;
    }while(left != right && left->next != right);
    return NULL;
}

// 指定值删除
bool del_value_list(DoubleList* list,TYPE data)
{
    Node* node = query_list(list,data);
    if(NULL == node)
        return false;

    _del_list(node);
    list->size--;
    return true;
}

// 修改指定位置的值
bool modify_index_list(DoubleList* list,int index,TYPE data)
{
    Node* node = _index_list(list,index);
    if(NULL == node)
        return false;

    node->data = data;
    return true;
}

// 修改指定值的值
bool modify_value_list(DoubleList* list,TYPE old,TYPE data)
{
    Node* node = query_list(list,old);
    if(NULL == node)
        return false;

    node->data = data;
    return true;
}


// 排序
void sort_list(DoubleList* list)
{
    /*
    // 冒泡排序
    bool flag = true;
    for(Node* r=list->tail->prev; flag && r!=list->head->next; r=r->prev)
    {
        flag = false;
        for(Node* l=list->head->next; l!=r; l=l->next)
        {
            if(l->data > l->next->data)
```

```
            {
                TYPE temp = l->data;
                l->data = l->next->data;
                l->next->data = temp;
                flag = true;
            }
        }
    }*/

    // 插入排序
    for(Node* i=list->head->next->next,*j; i!=list->tail; i=i->next)
    {
        TYPE data = i->data;
        for(j=i; j!=list->head->next && data < j->prev->data; j=j->prev)
        {
            j->data = j->prev->data;
        }
        j->data = data;
    }
}

int main(int argc,const char* argv[])
{
    DoubleList* list = create_list();
    for(int i=0; i<10; i++)
    {
        add_head_list(list,rand()%100);
    }
    sort_list(list);
    show_list(list);
    /*
    show_list(list);
    del_head_list(list);
    show_list(list);
    del_tail_list(list);
    show_list(list);
    */
    return 0;
}
```

6、双向循环链表

所谓的双向循环链表就是头节点的前驱是尾节点，头节点是尾节点的后继，这样的好处是可以让头节点、尾节点共用一个空白节点，有些奇怪的是当链表空时，空白节点的前驱和后继指针都指向自己，操作双向循环链表时，不存在空指针，以下是有修改代码，其它代码与双向链表没有区别。

```
// 创建节点
Node* create_node(TYPE data)
{
    Node* node = malloc(sizeof(Node));
    node->data = data;
    node->prev = node;
    node->next = node;
    return node;
}

typedef struct DoubleList
```

```
{
    Node* head;
    Node* tail;
    size_t size;
}DoubleList;

// 创建链表
DoubleList* create_list(void)
{
    TYPE data;
    DoubleList* list = malloc(sizeof(DoubleList));
    list->head = list->tail = create_node(data);
    list->size = 0;
    return list;
}

// 销毁链表
void destroy_list(DoubleList* list)
{
    Node* node = list->tail;
    do{
        Node* temp = node;
        node = node->prev;
        free(temp);
    }while(node!=list->head);
}
```

7、Linux内核链表

在Linux 内核中有一种通用的双向循环链表，链表要想做到通用，就无法知道节点中存储什么类型数据，所以就无法明确链表的节点。而面对这 样的问题Linux 内核中的链表的节点struct list_head只设计指针域，也就是节点中只有前驱、后续指针，Linux 内核中的链表只负责这些节点连接起来。

链表的使用者自己设计结构体，结构体中要包含struct list_head类型的节点成员，在链表接口时只需要提供节点成员的地址即可。

```
struct list_head {
    struct list_head *next, *prev;
};

typedef struct Student
{
    int id;
    char name[20];
    char sex;
    float score;
    struct list_head node;
}Student;

static inline void __list_add(struct list_head *new,
                  struct list_head *prev,
                  struct list_head *next)
{
    next->prev = new;
    new->next = next;
    new->prev = prev;
    prev->next = new;
```

```c
}

static inline void list_add(struct list_head *new, struct list_head *head)
{
    __list_add(new, head, head->next);
}

int main(int argc,const char* argv[])
{
    struct list_head* list = malloc(sizeof(struct list_head));
    INIT_LIST_HEAD(list);

    for(int i=0; i<10; i++)
    {
        Student* stup = malloc(sizeof(Student));
        stup->id = 10011+i;
        sprintf(stup->name,"hehe%d",i+1);
        stup->sex = rand()%2 ? 'w' :'m';
        stup->score = rand()%100;

        list_add(&stup->node,list);
    }
}
```

遍历链表时只能使用节点成员指针遍历，所以必须有一个工具，可以根据节点成员地址访问到它所在的结构体。

```c
#define list_entry(ptr, type, member) \
    ((type *)((char *)(ptr)-(unsigned long)(&((type *)0)->member)))

//1、假定0地址是结构体变量的地址，然后使用 -> 访问 struct list_head类型的成员，然后使用&地址计算出struct list_head类型的成员在结构体中的第几个字节，然后把地址转换成整数。
(unsigned long)(&((type *)0)->member)

//2、把struct list_head类型的成员的地址转换成进步值为1的char*类型，然后再减去struct list_head类型的成员到结构体开头的距离，就计算出结构变量首地址，然后再强制转换成结构体类型的地址
(char *)(ptr)
```

8、通用链表

Linux内核链表设计的很好，但是不利于初学者使用，另一种通用的设计思路就是借助void*兼容性，设计一种通用链表，但需要使用到回调函数。

```c
#ifndef LIST_H
#define LIST_H
#include <stdio.h>
#include <stdbool.h>

// 链表节点结构体
typedef struct Node
{
    void* ptr;
    struct Node* prev;
    struct Node* next;
}Node;
```

```
// 链表结构体
typedef struct List
{
    Node* head;
    size_t size;
}List;

// 创建节点
Node* create_node(void* ptr);

// 创建链表
List* create_list(void);

// 销毁链表
void destroy_list(List* list);

// 清理链表所有节点
void clear_list(List* list);

// 头添加
void add_list(List* list,void* ptr);

// 尾添加
void add_tail_list(List* list,void* ptr);

// 判断是否是空表
bool empty_list(List* list);

// 头删除
bool del_list(List* list);

// 尾删除
bool del_tail_list(List* list);

// 指定位置添加
bool inset_list(List* list,int index,void* ptr);

// 指定位置删除
bool del_index_list(List* list,int index);

// 访问指定位置的节点
void* get_list(List* list,int index);

// 指定值删除
bool del_value_list(List* list,void* ptr,int (*comper)(const void*,const void*));

// 查询
void* query_list(List* list,void* ptr,int (*comper)(const void*,const void*));
// 排序
void sort_list(List* list,void* ptr,int (*comper)(const void*,const void*));
// 遍历
void fput_list(List* list,void (*fput_ptr)(FILE*,const void*),FILE* fp);

#endif//LIST_H
```

```c
#include "list.h"
#include <stdlib.h>

static void _add_list(Node* node,Node* prev,Node* next)
{
    next->prev = node;
    node->next = next;
    node->prev = prev;
    prev->next = node;
}

static void _del_list(Node* node)
{
    node->prev->next = node->next;
    node->next->prev = node->prev;
    free(node);
}

static Node* _index_list(List* list,int index)
{
    if(index < list->size/2)
    {
        Node* node = list->head->next;
        for(int i=0; i<index; i++)
            node = node->next;
        return node;
    }

    Node* node = list->head->prev;
    for(int i=index; i<list->size-1; i++)
        node = node->prev;
    return node;
}

// 创建节点
Node* create_node(void* ptr)
{
    Node* node = malloc(sizeof(Node));
    node->ptr = ptr;
    node->next = node;
    node->prev = node;
    return node;
}

// 创建链表
List* create_list(void)
{
    List* list = malloc(sizeof(List));
    list->head = create_node(NULL);
    list->size = 0;
    return list;
}

// 销毁链表
void destroy_list(List* list)
{
```

```
    Node* node = list->head;
    do{
        Node* temp = node;
        node = node->next;
        free(temp);
    }while(node != list->head);

    free(list);
}

// 清理链表所有节点
void clear_list(List* list)
{
    Node* node = list->head->next;
    do{
        Node* temp = node;
        node = node->next;
        free(temp);
    }while(node != list->head);

    list->head->next = list->head;
    list->head->prev = list->head;
    list->size = 0;
}

// 头添加
void add_list(List* list,void* ptr)
{
    _add_list(create_node(ptr),list->head,list->head->next);
    list->size++;
}

// 尾添加
void add_tail_list(List* list,void* ptr)
{
    _add_list(create_node(ptr),list->head->prev,list->head);
    list->size++;
}

// 判断是否是空表
bool empty_list(List* list)
{
    return list->head->next == list->head;
}

// 头删除
bool del_list(List* list)
{
    if(empty_list(list))
        return false;

    _del_list(list->head->next);
    list->size--;
    return true;
}

// 尾删除
bool del_tail_list(List* list)
```

```
{
    if(empty_list(list))
        return false;

    _del_list(list->head->prev);
    list->size--;
    return true;
}

// 指定位置添加
bool inset_list(List* list,int index,void* ptr)
{
    if(index >= list->size)
        return false;

    Node* node = _index_list(list,index);
    _add_list(create_node(ptr),node->prev,node);
    list->size++;
    return true;
}

// 指定位置删除
bool del_index_list(List* list,int index)
{
    if(index >= list->size)
        return false;

    _del_list(_index_list(list,index));
    list->size--;
    return true;
}

// 指定值删除
bool del_value_list(List* list,void* ptr,int (*comper)(const void*,const void*))
{
    Node* node = list->head->next;
    while(node != list->head)
    {
        if(0 == comper(node->ptr,ptr))
        {
            _del_list(node);
            list->size--;
            return true;
        }
        node = node->next;
    }
    return false;
}

// 访问指定位置的节点
void* get_list(List* list,int index)
{
    if(index >= list->size)
        return NULL;

    return _index_list(list,index)->ptr;
}
```

```c

// 查询
void* query_list(List* list,void* ptr,int (*comper)(const void*,const void*))
{
    Node* node = list->head->next;
    while(node != list->head)
    {
        if(0 == comper(node->ptr,ptr))
            return node->ptr;
        node = node->next;
    }
}

// 排序
void sort_list(List* list,void* ptr,int (*comper)(const void*,const void*))
{
    bool flag = true;
    for(Node* r=list->head->prev; flag && r!=list->head->next; r=r->prev)
    {
        flag = false;
        for(Node* l=list->head->next; l!=r; l=l->next)
        {
            if(1 == comper(l->ptr,l->next->ptr))
            {
                void* tmp = l->ptr;
                l->ptr = l->next->ptr;
                l->next->ptr = tmp;
                flag = true;
            }
        }
    }
}

// 遍历
void fput_list(List* list,void (*fput_ptr)(FILE*,const void*),FILE* fp)
{
    Node* node = list->head->next;
    while(node != list->head)
    {
        fput_ptr(fp,node->ptr);
        node = node->next;
    }
}

#include <string.h>
typedef struct Admin{
    int id;
    char pwd[7];
}Admin;

int admin_login(const void* p1,const void* p2)
{
    Admin *a1 = (Admin*)p1,*a2 = (Admin*)p2;
    if(a1->id == a2->id && strcmp(a1->pwd,a2->pwd))
        return 0;
    return -1;
}
```

```c

int admin_del(const void* p1,const void* p2)
{
    Admin *a1 = (Admin*)p1,*a2 = (Admin*)p2;
    if(a1->id > a2->id)
        return 1;
    if(a1->id < a2->id)
        return -1;
    return 0;
}

void put_admin(FILE* fp,const void* ptr)
{
    Admin *admin = (Admin*)ptr;
    fprintf(fp,"%d %s\n",admin->id,admin->pwd);
}

int main()
{
    List* list = create_list();
    for(int i=0; i<10; i++)
    {
        Admin* admin = malloc(sizeof(Admin));
        admin->id = 10010+i;
        sprintf(admin->pwd,"123456");
        add_tail_list(list,admin);
    }

    Admin m = {10017,"123456"};
    del_value_list(list,&m,admin_del);

    /*
    Admin m = {10017,"123456"};
    if(NULL == query_list(list,&m,admin_login))
        printf("用户名或密码错误\n");
    else
        printf("登录成功!\n");
    */
    fput_list(list,put_admin,fopen("admin.txt","w"));
}
```

四、功能受限的线性表

> 栈和队列是两种重要的线性结构，从数据结构来看，他们也是线性表，其特殊性在于它们的基本操作是线性表的子集，也就中**功能受限的线性表**，也被称为限定性的数据结构。
>
> 但从数据类型角度来看，它们是和线性表不大相同，有些时候它们被当作一种管理数据的规则。

1、栈结构介绍

栈（stack) 是限定仅在表尾进行插入或删除操作和线性表（只有一端口能进出数据），对栈来说表尾和表头有特殊含义，表尾被称为**栈顶**，表头被称为**栈底**，没有元素的空表称为**空栈**，元素数量达到栈的容量称为**满栈**，数据添加到栈中叫**入栈、压栈**，数据从栈中删除叫**出栈、弹栈**，由于栈元素特殊添加和删除的规则，所以栈的元素会先进后出的现象，简称为**LIFO**（Last in first out）。

2、栈结构的顺序表示与实现

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define TYPE int

typedef struct StackArray
{
    TYPE* base;
    int top;
    size_t cap;
}StackArray;

// 创建栈
StackArray* create_stack(size_t cap)
{
    // 之所以使用成员指针是为了兼顾笔试题
    StackArray* stack = malloc(sizeof(StackArray));
    stack->base = malloc(sizeof(TYPE)*cap);
    stack->cap = cap;
    // 初始值值决定的栈空、栈满、入栈、查看栈顶时的top的操作,
    stack->top = -1;
    return stack;
}

// 销毁栈
void destroy_stack(StackArray* stack)
{
    free(stack->base);
    free(stack);
}

// 判断栈是否为空
bool empty_stack(StackArray* stack)
{
    // 因为top的初始值是-1
    return 0 > stack->top;
}

// 判断栈是否为满
bool full_stack(StackArray* stack)
{
    // 同上
    return stack->cap == stack->top+1;
}

// 入栈
bool push_stack(StackArray* stack,TYPE val)
{
```

```c
    if(full_stack(stack))
        return false;

    // 同上
    stack->base[++stack->top] = val;
    return true;
}

// 出栈
bool pop_stack(StackArray* stack)
{
    if(empty_stack(stack))
        return false;

    stack->top--;
    return true;
}

// 计算栈元素数量
size_t size_stack(StackArray* stack)
{
    return stack->top+1;
}

// 查看栈顶
bool top_stack(StackArray* stack,TYPE* ptr)
{
    if(empty_stack(stack))
        return false;

    *ptr = stack->base[stack->top];
    return true;
}

int main(int argc,const char* argv[])
{
    int num;
    StackArray* stack = create_stack(10);
    for(int i=0; i<10; i++)
    {
        push_stack(stack,i);
        if(top_stack(stack,&num))
            printf("top %d\n",num);
    }
    printf("----------------------\n");
    while(!empty_stack(stack))
    {
        top_stack(stack,&num);
        printf("top %d\n",num);
        pop_stack(stack);
    }
}
```

3、栈结构的链式表示与实现

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define TYPE int

typedef struct Node
{
    TYPE data;
    struct Node* next;
}Node;

Node* create_node(TYPE data)
{
    Node* node = malloc(sizeof(Node));
    node->data = data;
    node->next = NULL;
    return node;
}

typedef struct StackList
{
    Node* top;
    size_t cnt;
}StackList;

// 创建栈
StackList* create_stack(void)
{
    StackList* stack = malloc(sizeof(StackList));
    stack->top = NULL;
    stack->cnt = 0;
    return stack;
}

// 销毁栈
void destroy_stack(StackList* stack)
{
    while(NULL!=stack->top)
    {
        Node* node = stack->top;
        stack->top = node->next;
        free(node);
    }

    free(stack);
}

// 栈空
bool empty_stack(StackList* stack)
{
    return NULL == stack->top;
}

// 入栈
```

```c
void push_stack(StackList* stack,TYPE data)
{
    Node* node = create_node(data);
    node->next = stack->top;
    stack->top = node;
    stack->cnt++;
}

// 出栈
bool pop_stack(StackList* stack)
{
    if(empty_stack(stack))
        return false;

    Node* node = stack->top;
    stack->top = node->next;
    free(node);
    stack->cnt--;
    return true;
}

// 查看栈顶
TYPE top_stack(StackList* stack)
{
    return stack->top->data;
}

// 元素数量
size_t size_stack(StackList* stack)
{
    return stack->cnt;
}

int main(int argc,const char* argv[])
{
    StackList* stack = create_stack();
    for(int i=0; i<10; i++)
    {
        push_stack(stack,i);
        if(!empty_stack(stack))
            printf("top %d\n",top_stack(stack));
    }
    printf("-------------------\n");
    while(!empty_stack(stack))
    {
        printf("top %d\n",top_stack(stack));
        pop_stack(stack);
    }
}
```

4、栈的应用

1、内存管理，比如栈内存，它所以叫栈内存就是因为它遵循着栈的后进先出的规则，它支持着函数调用，函数在传参数就是把参数先压入到栈内存，等跳转过去后再把参数从栈内存弹出。

2、特殊的算法，例如：进制转换、表达式解析、迷宫求解。

5、队列介绍

它与栈刚好相反，是一种先进先出的线性表，它有两个端口添加、删除元素，一个端口只能添加元素，被称为入队，该端口被为队尾，另一个端口只能删除，被称为出队，该端口被称为队头。

6、队列结构的链式表示与实现

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define TYPE int

typedef struct Node
{
    TYPE data;
    struct Node* next;
}Node;

Node* create_node(TYPE data)
{
    Node* node = malloc(sizeof(Node));
    node->data = data;
    node->next = NULL;
    return node;
}

typedef struct QueueList
{
    Node* front;
    Node* rear;
}QueueList;

// 创建队列
QueueList* create_queue(void)
{
    QueueList* queue = malloc(sizeof(QueueList));
    queue->front = NULL;
    queue->rear = NULL;
    return queue;
}

// 销毁队列
void destroy_queue(QueueList* queue)
{
    while(NULL != queue->front)
    {
        Node* node = queue->front;
        queue->front = node->next;
        free(node);
    }
    free(queue);
}

// 判断队列是否为空
bool empty_queue(QueueList* queue)
{
```

```
    return NULL == queue->front;
}

// 入队
void push_queue(QueueList* queue,TYPE data)
{
    Node* node = create_node(data);
    if(empty_queue(queue))
    {
        queue->front = node;
        queue->rear = node;
        return;
    }

    queue->rear->next = node;
    queue->rear = node;
}

// 出队
bool pop_queue(QueueList* queue)
{
    if(empty_queue(queue))
        return false;

    Node* node = queue->front;
    queue->front = node->next;
    free(node);
}

// 查看队头元素，调用该函数前要先判断队列是否为空，否则就会使用到空指针
TYPE front_queue(QueueList* queue)
{
    return queue->front->data;
}

// 查看队尾元素，调用该函数前要先判断队列是否为空，否则就会使用到野指针
TYPE rear_queue(QueueList* queue)
{
    return queue->rear->data;
}

// 队列元素数量
size_t size_queue(QueueList* queue)
{
    size_t size = 0;
    for(Node* n=queue->front; NULL!=n; n=n->next)
    {
        size++;
    }
    return size;
}

int main(int argc,const char* argv[])
{
    QueueList* queue = create_queue();
    for(int i=0; i<10; i++)
    {
        push_queue(queue,i);
```

```
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
    }
    printf("--------------------\n");
    while(!empty_queue(queue))
    {
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
        pop_queue(queue);
    }
}
```

7、顺序队列的表示与实现

顺序栈的栈顶下标会随着元素入栈增大，元素出栈减小，从而让空间重复使用，而顺序队列的队头指针和队尾指针，会随着元素的入队出队一直增加，无法重复使用，而形成一次性的数据结构。

为了避免这种情况，当队头下标的队尾下标到达存储空间的末尾时，要想办法把队头下标的队尾下标"回头"，就是把顺序队列的存储空间当作一个"环形"，循环使用，所以顺序队列也叫循环队列。

带计数器版本的循环队列：

1、解决了计算元素数量的问题

2、解决了判断队列空、满状态的问题

3、缺点就是增加了一个成员，在创建、入队、出队时都要操作它。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define TYPE int

typedef struct QueueArray
{
    TYPE* base;
    int front;
    int rear;
    size_t cnt;
    size_t cap;
}QueueArray;

// 创建队列
QueueArray* create_queue(size_t cap)
{
    QueueArray* queue = malloc(sizeof(QueueArray));
    queue->base = malloc(sizeof(TYPE)*cap);
    queue->cap = cap;
    queue->front = 0;
    queue->rear = -1;
    queue->cnt = 0;
    return queue;
}

// 销毁队列
```

```c
void destroy_queue(QueueArray* queue)
{
    free(queue->base);
    free(queue);
}

// 判断队列是否为空
bool empty_queue(QueueArray* queue)
{
    return 0 == queue->cnt;
}

// 判断队列是否为满
bool full_queue(QueueArray* queue)
{
    return queue->cnt == queue->cap;
}

// 入队
bool push_queue(QueueArray* queue,TYPE data)
{
    if(full_queue(queue))
        return false;

    queue->rear = (queue->rear+1)%queue->cap;
    queue->base[queue->rear] = data;
    queue->cnt++;
    return true;
}

// 出队
bool pop_queue(QueueArray* queue)
{
    if(empty_queue(queue))
        return false;

    queue->front = (queue->front+1)%queue->cap;
    queue->cnt--;
    return true;
}

// 查看队头元素,判断队列是否为空
TYPE front_queue(QueueArray* queue)
{
    return queue->base[queue->front];
}

// 查看队尾元素
TYPE rear_queue(QueueArray* queue)
{
    return queue->base[queue->rear];
}

// 队列元素数量
size_t size_queue(QueueArray* queue)
{
    return queue->cnt;
}
```

```c

int main(int argc,const char* argv[])
{
    QueueArray* queue = create_queue(10);
    for(int i=0; i<10; i++)
    {
        push_queue(queue,i);
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
    }
    printf("---------------------\n");
    while(!empty_queue(queue))
    {
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
        pop_queue(queue);
    }
    printf("---------------------\n");
    for(int i=0; i<10; i++)
    {
        push_queue(queue,i);
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
    }
    printf("---------------------\n");
    while(!empty_queue(queue))
    {
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
        pop_queue(queue);
    }
    destroy_queue(queue);
    return 0;
}
```

不带计数器版本的循环队列：

1、如何判断队列空、状态?

把队头下标初始化为front=0，队尾下标也初始化rear=0，队列空时的判断条件是front == rear，但随机队列的使用，队列满时的判断也是front == rear。

解决方法就是把队列的最后一个位置空置不使用，这样队列满的状态front == rear+1;

2、如何计算队列元素数量?

(rear-front+cap)%cap

3、查看队尾元素

base[(read-1+cap)%cap]

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
```

```c

#define TYPE int

typedef struct QueueArray
{
    TYPE* base;
    int front;
    int rear;
    size_t cap;
}QueueArray;

// 创建队列
QueueArray* create_queue(size_t cap)
{
    QueueArray* queue = malloc(sizeof(QueueArray));
    queue->base = malloc(sizeof(TYPE)*cap+1);
    queue->cap = cap+1;
    queue->front = 0;
    queue->rear = 0;
    return queue;
}

// 销毁队列
void destroy_queue(QueueArray* queue)
{
    free(queue->base);
    free(queue);
}

// 判断队列是否为空
bool empty_queue(QueueArray* queue)
{
    return queue->front == queue->rear;
}

// 判断队列是否为满
bool full_queue(QueueArray* queue)
{
    return queue->front == (queue->rear+1)%queue->cap;
}

// 入队
bool push_queue(QueueArray* queue,TYPE data)
{
    if(full_queue(queue))
        return false;

    queue->base[queue->rear] = data;
    queue->rear = (queue->rear+1)%queue->cap;
    return true;
}

// 出队
bool pop_queue(QueueArray* queue)
{
    if(empty_queue(queue))
        return false;

```

```c
    queue->front = (queue->front+1)%queue->cap;
    return true;
}

// 查看队头元素,判断队列是否为空
TYPE front_queue(QueueArray* queue)
{
    return queue->base[queue->front];
}

// 查看队尾元素
TYPE rear_queue(QueueArray* queue)
{
    return queue->base[(queue->rear - 1 + queue->cap) % queue->cap];
}

// 队列元素数量
size_t size_queue(QueueArray* queue)
{
    return (queue->rear - queue->front + queue->cap) % queue->cap;
}

int main(int argc,const char* argv[])
{
    QueueArray* queue = create_queue(10);
    for(int i=0; i<10; i++)
    {
        push_queue(queue,i);
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
    }
    printf("---------------------\n");
    while(!empty_queue(queue))
    {
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
        pop_queue(queue);
    }
    printf("---------------------\n");
    for(int i=0; i<10; i++)
    {
        push_queue(queue,i);
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
    }
    printf("---------------------\n");
    while(!empty_queue(queue))
    {
        printf("front:%d rear:%d\n",
            front_queue(queue),
            rear_queue(queue));
        pop_queue(queue);
    }
    destroy_queue(queue);
}
```

8、队列的应用

队列结构一般应用于业务处理，例如：银行叫号系统，12306购票系统，电商的订单处理等。

五、树型结构

1、树的介绍

树是一种非线性的数据结构，是以分支关系定义的层次结构，比如人类社会中的族谱、及各种机制、组织的关系都可以用树形象的表示。重点学习二叉树的存储和相关操作，还要讨论树、森林、二叉树的转换关系。

2、树的定义和基本术语

树：是n个结点的有限集，当0==n时称为空树，我们不讨论空树。

根结点：树的最顶层的结点，一棵树有且仅有一个。

子树：一棵树除根结点外，剩余的是若干个互不相交的有限集，每一个集合本身又是棵树，称称为根的子树。

结点的度：树的结点包含一个数据元素及若干个指向其子树的分支，结点拥有的子树称为结点的度。

叶子结点：结构的度为0，被称为叶子结点或终端结点。

分支结点：结构的度不为0，被称为分支结点或非终端结点，也被称为内部结点。

树的度：是指树内各结点度的最大值。

密度：指的是一棵树中，所有结点的总数。

孩子、双亲、兄弟、祖先、子孙：结点的子树称为该结点的孩子，而该结点是孩子结点的双亲，拥有共同双亲的结点互为兄弟，从双亲结点往上，直到根结点都称为孩子结点的祖先结点，以某结点为根的子树中的任一结点都被称为该结点的子孙。

层数、深度、高度：从根结点开始定义，根为第一层、根的孩子为第二层依次类推，树中结点的最大层数被称为树的深度或高度，双亲在同一层的结点互为堂兄弟。

有序树和无序树：将树中结点的各子树看成从左到右是有序次，即不能交换（顺序有意义，表达一些含义），则称该树为有序树，否则称为无序树。

森林：若干个棵互不相交的树的集合称为森林，对树中每个结点而言，其子树集合就是森林。

就逻辑结构而言，任何一棵树都是一个二元组 Tree = (root,F)，其中root是数据元素，称做树的根结点，F是若干棵子树构成的森林。

3、二叉树的定义和性质

二叉树：

是一种特殊的树型结构，也就是每个结点最多有两棵子树（二叉树中不存在度大于2的结点），并且二叉树的子树有左右之分，顺序不能颠倒。

满二叉树：

若一棵树的层数为k，它总结点数是2^k-1，则这棵树就是满二叉树。

完全二叉树：

若一棵树的层数为k，它前k-1层的总结点数是2^(k-1)-1，第k层的结点按照从左往右的顺序排列，则这棵树就是完全二叉树。

二叉树的性质：

性质1：在二叉树的第i层上，最多有2^(i-1)个结点。

性质2：深夜为k的二叉树，最多有2^k-1个节点。

性质3：对于任何一棵二叉树，如果叶子结点的数量为n0，度为2结点的数量为n2，则n0=n2+1；

性质4：具有n个结点的完全二叉树的高度为(log2n)+1。

性质5：有一个n个结点的完全二叉树，结点按照从上到下从左到右的顺序排序为1~n。

1、i > 1时，i/2就是它的双亲结点。

2、i*2是i的左子树，当i*2>n时，则i没有左子树。

3、2*i+1是i的右子树，2*i+1>n时，则i没有右子树。

4、二叉树的遍历

前序遍历：

1、判断二叉树是否为空，若二叉树为空，则不操作。

2、访问根结点

3、前序遍历左子树

4、前序遍历右子树

中序遍历：

1、判断二叉树是否为空，若二叉树为空，则不操作。

2、中序遍历左子树

3、访问根结点

4、中序遍历右子树

后序遍历：

1、判断二叉树是否为空，若二叉树为空，则不操作。

2、后序遍历左子树

3、后序遍历右子树

4、访问根结点

层序遍历：

按照从上到下、从左到右的顺序遍历二叉树，需要与队列结构配合，普通的顺序队列即可，不需要链式队列或循环队列。

5、二叉树的顺序表示与实现

前提：由于顺序存储需要根据元素的相对位置确定关系，所有先把二叉树补成完全二叉树，之前空的点可以使用特殊的值表示，并把完全二叉树的层序遍历结果存储到数组中，只有补成了完全二叉树，才能根据性质5的公式对二叉树进行相关操作。

注意：性质5的公式是按照结点的序号设计的，在使用时要注意数组下标与序号的转换。

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <math.h>

#define PARENT_TREE(i)  ((i + 1) / 2 - 1)
#define LEFT_TREE(i)    ((i + 1) * 2 - 1)
#define RIGHT_TREE(i)   ((i + 1) * 2)

typedef struct TreeArray
{
    char *arr;
    size_t cnt;
} TreeArray;

// 创建树
TreeArray *create_tree(const char *str)
{
    TreeArray *tree = malloc(sizeof(TreeArray));
    tree->cnt = strlen(str);
    tree->arr = malloc(tree->cnt);
    strncpy(tree->arr, str, tree->cnt);
    return tree;
}

// 初始化树
void init_tree(TreeArray *tree, const char *str)
{
    free(tree->arr);
    tree->cnt = strlen(str);
    tree->arr = malloc(tree->cnt);
    strncpy(tree->arr, str, tree->cnt);
}

// 销毁树
void destroy_tree(TreeArray *tree)
{
    free(tree->arr);
    free(tree);
}

// 清空树
void clear_tree(TreeArray *tree)
{
    free(tree->arr);
    tree->cnt = 0;
}

```

```c
// 判断是否是空树
bool empty_tree(TreeArray *tree)
{
    return 0 == tree->cnt || '#' == tree->arr[0];
}

// 计算树的深度
size_t hight_tree(TreeArray *tree)
{
    size_t hight = 1;
    while (pow(2, hight) < tree->cnt)
        hight++;
    return hight;
}

// 访问根结点
char root_tree(TreeArray *tree)
{
    if (empty_tree(tree))
        return '#';

    return tree->arr[0];
}

// 访问指定的结点
char value_tree(TreeArray *tree, size_t index)
{
    if (index >= tree->cnt)
        return '#';

    return tree->arr[index];
}

// 给指定的位置赋值
bool assign_tree(TreeArray *tree, size_t index, char value)
{
    if (index >= tree->cnt)
        return false;

    tree->arr[index] = value;
    return true;
}

// 访问双亲结点
char parent_tree(TreeArray *tree, size_t index)
{
    if (0 == index || PARENT_TREE(index) >= tree->cnt)
        return '#';

    return tree->arr[PARENT_TREE(index)];
}

// 访问左子树结点
char left_child_tree(TreeArray *tree, size_t index)
{
    if (LEFT_TREE(index) >= tree->cnt)
        return '#';

```

```c
    return tree->arr[LEFT_TREE(index)];
}

// 访问右子树结点
char right_child_tree(TreeArray *tree, size_t index)
{
    if (RIGHT_TREE(index) >= tree->cnt)
        return '#';
    return tree->arr[RIGHT_TREE(index)];
}

// 访问左兄弟结点
char left_sigling_tree(TreeArray *tree, size_t index)
{
    int left = LEFT_TREE(PARENT_TREE(index));
    if (0 == index || left >= tree->cnt || left == index)
        return '#';

    return tree->arr[left];
}

// 访问右兄弟结点
char right_sigling_tree(TreeArray *tree, size_t index)
{
    int right = RIGHT_TREE(PARENT_TREE(index));
    if (0 == index || right >= tree->cnt || right == index)
        return '#';

    return tree->arr[right];
}

// 插入子树 lr(left true,right false)
bool insert_tree(TreeArray *tree, size_t index, bool lr, char value)
{
    size_t pos = lr ? LEFT_TREE(index) : RIGHT_TREE(index);
    if (pos >= tree->cnt || '#' != tree->arr[pos])
        return false;

    tree->arr[pos] = value;
    return true;
}

bool delete_tree(TreeArray *tree, size_t index, bool lr)
{
    // 计算出要删除的子树的下标
    size_t pos = lr ? LEFT_TREE(index) : RIGHT_TREE(index);
    // 判断要删除的子树下标是否合法
    if (pos >= tree->cnt)
        return false;

    // 计算出要删除结点的左右子树下标
    size_t pos_left = LEFT_TREE(pos), pos_right = RIGHT_TREE(pos);
    // 判断删除的结点是否有左子树，如果有则删除失败
    if (pos_left < tree->cnt && '#' != tree->arr[pos_left])
        return false;
    // 判断删除的结点是否有右子树，如果有则删除失败
    if (pos_right < tree->cnt && '#' != tree->arr[pos_right])
        return false;
```

```c

    tree->arr[pos] = '#';
    return true;
}

// 前序遍历二叉树
void _pre_order_tree(TreeArray *tree, int index)
{
    // 根
    if (index >= tree->cnt || '#' == tree->arr[index])
        return;
    printf("%c ", tree->arr[index]);
    // 左
    _pre_order_tree(tree, LEFT_TREE(index));
    // 右
    _pre_order_tree(tree, RIGHT_TREE(index));
}

// 前序遍历
void pre_order_tree(TreeArray *tree)
{
    _pre_order_tree(tree, 0);
    printf("\n");
}

void _in_order_tree(TreeArray *tree, int index)
{
    if (index >= tree->cnt || '#' == tree->arr[index])
        return;
    // 左
    _in_order_tree(tree, LEFT_TREE(index));
    // 根
    printf("%c ", tree->arr[index]);
    // 右
    _in_order_tree(tree, RIGHT_TREE(index));
}

// 中序遍历
void in_order_tree(TreeArray *tree)
{
    _in_order_tree(tree, 0);
    printf("\n");
}

void _post_order_tree(TreeArray *tree, int index)
{
    if (index >= tree->cnt || '#' == tree->arr[index])
        return;
    // 左
    _post_order_tree(tree, LEFT_TREE(index));
    // 右
    _post_order_tree(tree, RIGHT_TREE(index));
    // 根
    printf("%c ", tree->arr[index]);
}

// 后序遍历
void post_order_tree(TreeArray *tree)
```

```c
{
    _post_order_tree(tree, 0);
    printf("\n");
}

// 层序遍历
void level_order_tree(TreeArray *tree)
{
    for (int i = 0; i < tree->cnt; i++)
    {
        if ('#' != tree->arr[i])
            printf("%c ", tree->arr[i]);
    }
    printf("\n");
}

int main(int argc, const char *argv[])
{
    TreeArray *tree = create_tree("FCEADHG##B###M");
    level_order_tree(tree);
    printf("--------------------\n");
    printf("%d\n", hight_tree(tree));
    printf("%c\n", right_sigling_tree(tree, 4));
    insert_tree(tree, 0, true, 'X');
    printf("%d\n", delete_tree(tree, 1, false));
    return 0;
}
```

6、二叉树的链式表示与实现

两种创建链式二叉树的方式：

方式一：需要给每个度不为2的结点补充一些空白子结点，使用树中除了空白结点其它结点的度全为2，然后以前序的方式遍历二叉树，然后以遍历的结果创建二叉树。

方式二：不需要对二叉做任何改变，以前序+中序或中序+后序的遍历结果创建二叉树。

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct TreeNode
{
    char data;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

TreeNode *create_node(char data)
{
    TreeNode *node = malloc(sizeof(TreeNode));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// 方式1
```

```c
TreeNode *create_tree(const char **str)
{
    if ('#' == **str)
        return NULL;
    TreeNode *root = create_node(**str);
    *str += 1;
    root->left = create_tree(str);
    *str += 1;
    root->right = create_tree(str);
    return root;
}

// 方式二
TreeNode *pre_in_create_tree(char *pre, char *in, int len)
{
    if (len <= 0)
        return NULL;

    TreeNode *root = create_node(*pre);

    int pos = 0;
    while (*pre != in[pos])
        pos++;

    root->left = pre_in_create_tree(pre + 1, in, pos);
    root->right = pre_in_create_tree(pre + 1 + pos, in + 1 + pos, len - 1 - pos);
    return root;
}

// 方式二
TreeNode *post_in_create_tree(char *post, char *in, int len)
{
    if (len <= 0)
        return NULL;

    TreeNode *root = create_node(post[len - 1]);

    int pos = 0;
    while (post[len - 1] != in[pos])
        pos++;

    root->left = post_in_create_tree(post, in, pos);
    root->right = post_in_create_tree(post + pos, in + pos + 1, len - 1 - pos);
}

size_t height_tree(TreeNode *root)
{
    if (NULL == root)
        return 0;

    size_t lh = height_tree(root->left);
    size_t rh = height_tree(root->right);

    return lh > rh ? lh + 1 : rh + 1;
}

```

```c
size_t density_tree(TreeNode *root)
{
    if (NULL == root)
        return 0;

    return 1 + density_tree(root->left) + density_tree(root->right);
}

void pre_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;
    printf("%c ", root->data);
    pre_order_tree(root->left);
    pre_order_tree(root->right);
}

void in_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    in_order_tree(root->left);
    printf("%c ", root->data);
    in_order_tree(root->right);
}

void post_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    post_order_tree(root->left);
    post_order_tree(root->right);
    printf("%c ", root->data);
}

void level_tree(TreeNode *root)
{
    size_t density = density_tree(root), front = 0, rear = 0;
    TreeNode *queue[density];
    queue[rear++] = root;

    while (front != rear)
    {
        TreeNode *node = queue[front++];
        printf("%c ", node->data);
        if (NULL != node->left)
            queue[rear++] = node->left;
        if (NULL != node->right)
            queue[rear++] = node->right;
    }
}

int main(int argc, const char *argv[])
{
    const char *pre_str = "FCA##DB###EH##GM###";
    TreeNode *pre_root = create_tree(&pre_str);
```

```
    pre_order_tree(pre_root);
    printf("\n");

    /*
    char post[] = "ADCHGEF";
    char in[] = "ACDFHEG";
    TreeNode* root = post_in_create_tree(post,in,7);
    pre_order_tree(root);
    printf("%d\n",height_tree(root));
    printf("%d\n",density_tree(root));
    */
    level_tree(pre_root);

    return 0;
}
```

7、二叉搜索树的表示与实现

二叉搜索树介绍：

二叉搜索树又叫作有序二叉树、二叉查找树，具有以下特点：若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值； 若它的右子树不空，则右子树上所有结点的值均大于它的根结点的值；它的左、右子树也分别为二叉搜索树。

二叉搜索树作为一种经典的数据结构，它既有链表的快速插入与删除操作的特点，又有数组快速查找的优势（O(log2n)；所以应用十分广泛，例如在文件系统和数据库系统一般会采用这种数据结构进行高效率的排序与检索操作。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct TreeNode
{
    int val;
    struct TreeNode *left;
    struct TreeNode *right;
    size_t height;  // 结点的高度
    size_t density; // 结点的密度
} TreeNode;

// 创建结点
TreeNode *create_node(int val)
{
    TreeNode *root = malloc(sizeof(TreeNode));
    root->val = val;
    root->left = NULL;
    root->right = NULL;
    root->height = 1;
    root->density = 1;
    return root;
}

// 自动调整树的高度和密度
void auto_tree(TreeNode *root)
```

```
{
    if (NULL == root)
        return;

    int lh = 0, rh = 0, ld = 0, rd = 0;
    if (NULL != root->left)
    {
        // 当左子树不为空时，调整左子树的高度和密度
        auto_tree(root->left);
        // 重新获取左子树的高度和密度
        lh = root->left->height;
        ld = root->left->density;
    }

    if (NULL != root->right)
    {

        // 当右子树不为空时，调整右子树的高度和密度
        auto_tree(root->right);
        // 重新获取右子树的高度和密度
        rh = root->right->height;
        rd = root->right->density;
    }

    // 更新当前结点的高度和密度
    root->height = lh > rh ? lh + 1 : rh + 1;
    root->density = ld + rd + 1;
}

void _add_tree(TreeNode **root, TreeNode *node)
{
    if (NULL == *root)
    {
        *root = node;
        return;
    }

    if ((*root)->val > node->val)
        _add_tree(&(*root)->left, node);
    else
        _add_tree(&(*root)->right, node);
}

void add_tree(TreeNode **root, int val)
{
    // root是main函数中root指针的地址
    _add_tree(root, create_node(val));
    auto_tree(*root);
}

// 之所这样设计是为了方便删除并兼顾查询，查询到的是指向值为val结点的指针的地址
TreeNode **_query_tree(TreeNode **root, int val)
{
    if (NULL == *root)
        return root;

    if (val == (*root)->val)
        return root;
```

```
    if (val <= (*root)->val)
        return _query_tree(&(*root)->left, val);
    else
        return _query_tree(&(*root)->right, val);
}

bool query_tree(TreeNode *root, int val)
{
    TreeNode **node = _query_tree(&root, val);
    return NULL != *node;
}

void _del_tree(TreeNode **root)
{
    TreeNode *temp = *root;
    if (NULL == temp->left)
        *root = temp->right;
    else if (NULL == temp->right)
        *root = temp->left;
    else
    {
        *root = temp->left;
        _add_tree(root, temp->right);
    }
    free(temp);
}

bool del_tree(TreeNode **root, int val)
{
    TreeNode **node = _query_tree(root, val);
    if (NULL == *node)
        return false;

    _del_tree(node);
    auto_tree(*root);
    return true;
}

bool modify_tree(TreeNode **root, int old, int val)
{
    TreeNode **node = _query_tree(root, old);
    if (NULL == *node)
        return false;

    _del_tree(node);
    _add_tree(root, create_node(val));
    auto_tree(*root);
    return true;
}

void _in_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    _in_order_tree(root->left);
    printf("%d ", root->val);
    _in_order_tree(root->right);
```

```c
}

void in_order_tree(TreeNode *root)
{
    _in_order_tree(root);
    printf("\n");
}

void level_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    TreeNode *queue[root->density];
    int front = 0, rear = 0;
    queue[rear++] = root;

    while (front != rear)
    {
        int cnt = rear - front;
        while (cnt--)
        {
            TreeNode *node = queue[front++];
            printf("[v:%d h:%d d:%d] ", node->val, node->height, node->density);
            if (NULL != node->left)
                queue[rear++] = node->left;
            if (NULL != node->right)
                queue[rear++] = node->right;
        }
        printf("\n");
    }
}

// 访问第k个小的数，按大小值的顺序访问
void _assign_tree(TreeNode *root, int k, int *pos, int *ptr)
{
    if (NULL == root)
        return;

    _assign_tree(root->left, k, pos, ptr);
    if (++*pos == k)
        *ptr = root->val;
    _assign_tree(root->right, k, pos, ptr);
}

int assign_tree(TreeNode *root, int k)
{
    if (k < 1)
        return -1;

    int pos = 0, val = -1;
    _assign_tree(root, k, &pos, &val);
    return val;
}

int main(int argc, const char *argv[])
{
```

```
    TreeNode *root = NULL;
    for (int i = 0; i < 10; i++)
    {
        add_tree(&root, rand() % 100);
    }
    in_order_tree(root);
    modify_tree(&root, 77, 99);
    in_order_tree(root);
    level_order_tree(root);

    return 0;
}

```

二叉搜索树的缺点：

二叉搜索树的元素添加顺序会影响二叉搜索树的形状，二叉搜索树的形状会影响它的操作效率，在极端情况下，二叉搜索树可能会呈单枝状分布，使用速度接近单向链表，这种极端情况出现在的原因就添加的元素基本有序。

8、平衡二叉树

平衡二叉树介绍：

平衡二叉树的全称叫平衡二叉搜索树，它首是一棵二叉搜索树，且具有以下性质：它的左右两个子树的高度差的绝对值不超过1，并且左右两个子树都是一棵平衡二叉树。

对于一般的二叉搜索树，它的理想高度是为log2n，其各操作的时间复杂度(O(log2n))。但是，在某些极端的情况下(如：在插入的序列是有序的时)，二叉搜索树将退化成近似单向链表，此时它的操作时间复杂度将退化成线性的，即O(n)。

我们可以通过随机化建立二叉搜索树来尽量的避免这种情况，但是在进行了多次的操作之后，由于在删除时，我们总是选择将待删除节点的后继代替它本身，这样就会造成总是右边的节点数目减少，以至于树向左偏沉。这同时也会造成树的平衡性受到破坏，提高它的操作的时间复杂度，而平衡二叉搜索树就可以弥补这个缺点。

如何把不平衡的二叉树调整为平衡二叉树：

我在检查二叉树是否平衡时，肯定是从叶子结点往根结点遍历，因此我们遇到的不平衡的二叉树有以下四种情况，分别是：

```
//1 解决方法：以B轴向右旋转
      A                          B
     / \                       /   \
    B   t1                    C     A
   / \                       / \   /  \
  C   t2                    t4 t3 t2  t1
 / \
t4   t3


//2 解决方法：先以C轴向左旋转，再以C为轴向右旋转
    A                 A                  C
   / \               / \               /   \
  B   t1            C   t1            B      A
 / \               / \               /  \   /  \
t2  C             B   t3            t2  t4 t3   t1
   / \           / \
```

```
  t4   t3      t2  t4

//3 解决方法：以B轴向左旋转
  A               B
 / \            /   \
t1  B          A     C
   / \        / \   / \
  t2  C      t1 t2 t3  t4
     / \
    t3   t4

//4 解决方法：以C轴向右旋转，再以C轴向左旋转
  A            A               C
 / \          / \            /   \
t1  B        t1  C          A     B
   / \          / \        / \   / \
  C   t4       t2  B      t1 t2 t3  t4
 / \               / \
t2   t3           T3  T4
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define TreeHight(node) (NULL == (node) ? 0 : (node)->height)

typedef struct TreeNode
{
    int val;
    struct TreeNode *left;
    struct TreeNode *right;
    size_t height;
} TreeNode;

// 创建结点
TreeNode *create_node(int val)
{
    TreeNode *root = malloc(sizeof(TreeNode));
    root->val = val;
    root->left = NULL;
    root->right = NULL;
    root->height = 1;
    return root;
}

// 调整树的高度
void auto_height_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    auto_height_tree(root->left);
    auto_height_tree(root->right);
    int lh = TreeHight(root->left);
    int rh = TreeHight(root->right);
    root->height = lh > rh ? lh + 1 : rh + 1;
}
```

```

void right_rotate_tree(TreeNode **rpp)
{
    TreeNode *A = *rpp;
    TreeNode *B = A->left;
    TreeNode *t = B->right;

    *rpp = B;
    B->right = A;
    A->left = t;
    auto_height_tree(*rpp);
}

void left_rotate_tree(TreeNode **rpp)
{
    TreeNode *A = *rpp;
    TreeNode *B = A->right;
    TreeNode *t = B->left;

    *rpp = B;
    B->left = A;
    A->right = t;
    auto_height_tree(*rpp);
}

void auto_balance_tree(TreeNode **rpp)
{
    if (NULL == *rpp)
        return;

    auto_balance_tree(&(*rpp)->left);
    auto_balance_tree(&(*rpp)->right);

    int lh = TreeHight((*rpp)->left);
    int rh = TreeHight((*rpp)->right);

    if (lh - rh > 1)
    {
        if (TreeHight((*rpp)->left->left) >= TreeHight((*rpp)->left->right))
            right_rotate_tree(rpp);
        else
        {
            left_rotate_tree(&(*rpp)->left);
            right_rotate_tree(rpp);
        }
        auto_balance_tree(rpp);
    }
    else if (lh - rh < -1)
    {
        if (TreeHight((*rpp)->right->right) >= TreeHight((*rpp)->right->left))
            left_rotate_tree(rpp);
        else
        {
            right_rotate_tree(&(*rpp)->right);
            left_rotate_tree(rpp);
        }
```

```
        auto_balance_tree(rpp);
    }
}

bool is_balance_tree(TreeNode *root)
{
    if (NULL == root)
        return true;

    bool flag1 = is_balance_tree(root->left);
    bool flag2 = is_balance_tree(root->right);

    return flag1 && flag2 && 1 >= abs(TreeHight(root->left) - TreeHight(root->right));
}

void _in_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    _in_order_tree(root->left);
    printf("%d ", root->val);
    _in_order_tree(root->right);
}

void in_order_tree(TreeNode *root)
{
    _in_order_tree(root);
    printf("\n");
}

void level_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    TreeNode *queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;

    while (front != rear)
    {
        int cnt = rear - front;
        while (cnt--)
        {
            TreeNode *node = queue[front++];
            printf("[v:%d h:%d] ", node->val, node->height);
            if (NULL != node->left)
                queue[rear++] = node->left;
            if (NULL != node->right)
                queue[rear++] = node->right;
        }
        printf("\n");
    }
}

// 添加结点
```

```c
void _add_tree(TreeNode **rpp, TreeNode *node)
{
    if (NULL == *rpp)
    {
        *rpp = node;
        return;
    }

    if (node->val < (*rpp)->val)
        _add_tree(&(*rpp)->left, node);
    else
        _add_tree(&(*rpp)->right, node);
}

// 添加元素
void add_tree(TreeNode **rpp, int val)
{
    // 创建结点并添加
    _add_tree(rpp, create_node(val));
    // 调整树的高度
    auto_height_tree(*rpp);
    // 让树自动平衡
    auto_balance_tree(rpp);
}

// 初始化二权树
TreeNode *init_tree(int *arr, int size)
{
    TreeNode *root = NULL;
    // 循环把所有的元素创建结点并添加到树中
    for (int i = 0; i < size; i++)
    {
        _add_tree(&root, create_node(arr[i]));
    }

    // 调整树的高度
    auto_height_tree(root);
    // 让树自动平衡
    auto_balance_tree(&root);
    return root;
}

int main(int argc, const char *argv[])
{
    int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    TreeNode *root = NULL;
    for (int i = 0; i < 10; i++)
    {
        add_tree(&root, arr[i]);
    }
    printf("%d\n", is_balance_tree(root));
    return 0;
}
```

平衡二叉树的优点：

避免二叉搜索树的单支状分布的情况，能让二叉搜索树以最佳的状态进行操作，时间复杂接近O(log2n)。

平衡二叉树的缺点：

平衡二叉树在创建、添加、删除时，为了让二叉树保持平衡需要进行大量的左旋、右旋、高度计算，所以平衡二叉树在创建时，添加、删除结点时速度比较慢。

因此平衡二叉树适合使用在数据量大且数据量稳定的情况，没有大量的添加、删除动作，大多数情况下都是进行查询操作。

红黑树与AVL树的区别：

1、AVL树在物理上就是平衡的，所以在创建、添加、删除时速度比较慢，但它的查询速度接近平衡二叉树的极限。

2、红黑树是一种特殊的AVL树，它的物理结构不是严格的平衡，而是接近平衡，从根结点所有的叶子结点的速度大致相同，它的非绝对平衡使用节约很多左旋、右旋的次数，因此它创建、添加、删除时速度比AVL树要快，查询速度接近AVL树。

9、线索二叉树

线索二叉树介绍：

我们在遍历二叉树时会得到一个线性的序列结果，但遍历的过程是非线性的(函数递归)，该过程比较耗时，我们可以借助结点中的空的指针，增加一些线索，使用二叉树能够使用循环语句进行遍历，提高二叉树遍历的速度。

添加线索的方法：

在二叉树的结点中增加两个成员，使它的结点成为以下结构：

```
// 使用这种结点可以构建二叉双向链表，作为二叉树的存储结点，也叫作线索链表
typedef struct TreeNode
{
    int val;                // 数据域
    bool ltag;              // false == ltag left指针域指向是当前结点的左子树，否则指向的是它的前驱结点
    struct TreeNode* left;
    bool rtag;              // false == rtag right指针域指向是当前结点的右子树，否则指向的是它的后继结点
    struct TreeNode* right;
}
```

线索二叉树的表示与实现：

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define Link false
#define Thread true

typedef struct TreeNode
{
    int val;
```

```c
    bool ltag;
    struct TreeNode *left;
    bool rtag;
    struct TreeNode *right;
} TreeNode;

TreeNode *create_node(int val)
{
    TreeNode *root = malloc(sizeof(TreeNode));
    root->val = val;
    root->ltag = false;
    root->left = NULL;
    root->rtag = false;
    root->right = NULL;
    return root;
}

void _add_tree(TreeNode **rpp, TreeNode *node)
{
    if (NULL == *rpp)
    {
        *rpp = node;
        return;
    }

    if (node->val < (*rpp)->val)
        _add_tree(&(*rpp)->left, node);
    else
        _add_tree(&(*rpp)->right, node);
}

void add_tree(TreeNode **rpp, int val)
{
    _add_tree(rpp, create_node(val));
}

TreeNode *init_tree(int *arr, size_t len)
{
    TreeNode *root = NULL;
    for (int i = 0; i < len; i++)
    {
        _add_tree(&root, create_node(arr[i]));
    }
    return root;
}

void _in_order_tree(TreeNode *root)
{
    if (NULL == root)
        return;

    _in_order_tree(root->left);
    printf("%d ", root->val);
    _in_order_tree(root->right);
}

void in_order_tree(TreeNode *root)
{
```

```c
    _in_order_tree(root);
    printf("\n");
}

static TreeNode *pre;
void in_threading(TreeNode *root)
{
    if (NULL == root)
        return;

    in_threading(root->left);
    if (NULL == root->left)
    {
        root->ltag = Thread;
        root->left = pre;
    }
    if (NULL == pre->right)
    {
        pre->rtag = Thread;
        pre->right = root;
    }
    pre = root;
    in_threading(root->right);
}

TreeNode *in_order_threading(TreeNode *root)
{
    TreeNode *head = create_node(-1);
    head->ltag = Link;
    head->rtag = Thread;
    head->right = head;

    if (NULL == root)
        head->left = head;
    else
    {
        head->left = root;
        pre = head;
        in_threading(root);
        pre->right = head;
        pre->rtag = Thread;
    }
    return head;
}

void in_order_thread_tree(TreeNode *head)
{
    TreeNode *node = head->left;
    while (node != head)
    {
        while (Link == node->ltag)
            node = node->left;
        printf("%d ", node->val);
        while (node->rtag == Thread && node->right != head)
        {
            node = node->right;
            printf("%d ", node->val);
        }
```

```
        node = node->right;
    }
}

int main(int argc, const char *argv[])
{
    int arr[10];
    for (int i = 0; i < 10; i++)
    {
        arr[i] = rand() % 100;
    }

    TreeNode *root = init_tree(arr, 10);
    in_order_tree(root);

    // 添加线索
    TreeNode *inhead = in_order_threading(root);
    // 遍历线索二叉树
    in_order_thread_tree(inhead);
    return 0;
}
```

10、堆树

堆结构介绍：

大根堆：根结点的值比它的左右子树都要大，同时它的左右子树也遵循这项规则，也就是说一棵树的根结点中存储的是这棵树中的最大值，这种完全二叉树叫大根堆。

小根堆：根结点的值比它的左右子树都要小，同时它的左右子树也遵循这项规则，也就是说一棵树的根结点中存储的是这棵树中的最小值，这种完全二叉树叫小根堆。

堆结构是是一种特殊的完全二叉树，它与堆栈完全不同且没有任何关系，我们可以从堆中顺序获取堆顶的数据，也可以使用大根堆、小根堆的规则可以对顺序表进行排序。

堆结构的存储：

堆结构是完全二叉树，所以特别适合顺序表存储，因此要熟练掌握二叉树性质5的分式：

> 有一个n个结点的完全二叉树，结点按照从上到下从左到右的顺序排序为1~n。
>
> 1、i > 1时，i/2就是它的双亲结点。
>
> 2、i*2是i的左子树，当i*2>n时，则i没有左子树。
>
> 3、2*i+1是i的右子树，2*i+1>n时，则i没有右子树。

堆结构的表示与实现：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define PARENT(index) ((index + 1) / 2 - 1)
#define LEFT_CHILD(index) ((index + 1) * 2 - 1)
#define RIGHT_CHILD(index) ((index + 1) * 2)

#define swap_mem(p1, p2, size)  \
```

```c
    do                          \
    {                           \
        char temp[size];        \
        memcpy(temp, p1, size); \
        memcpy(p1, p2, size);   \
        memcpy(p2, temp, size); \
    } while (0)

#define swap(a, b) swap_mem(&a, &b, sizeof(a))

typedef struct MaxHeap
{
    int *base;
    size_t cnt;
    size_t cal;
} MaxHeap;

// 把base中的数据调整为堆结构
void adjust_max_heap(MaxHeap *heap, int index)
{
    // 从index下标处，向上调整堆结构
    while (index > 0)
    {
        // 计算出双亲下标
        int parent = PARENT(index);
        // 计算出双亲结点的左子树
        int left = LEFT_CHILD(parent);
        // 如果左子树下标合法并且值大于双亲，则与双亲交换
        if (left < heap->cnt && heap->base[left] > heap->base[parent])
            swap(heap->base[left], heap->base[parent]);

        int right = RIGHT_CHILD(parent);
        // 如果右子树下标合法并且值大于双亲，则与双亲交换
        if (right < heap->cnt && heap->base[right] > heap->base[parent])
            swap(heap->base[right], heap->base[parent]);

        // 向上一层
        index = parent;
    }
}

// 创建一个空堆
MaxHeap *create_max_heap(size_t cal)
{
    MaxHeap *heap = malloc(sizeof(MaxHeap));
    heap->base = malloc(sizeof(int) * cal);
    heap->cal = cal;
    heap->cnt = 0;
    return heap;
}

// 初始化堆
MaxHeap *init_max_heap(int *arr, size_t len)
{
    MaxHeap *heap = malloc(sizeof(MaxHeap));
    heap->base = malloc(sizeof(int) * len);
    memcpy(heap->base, arr, sizeof(int) * len);
    heap->cal = len;
```

```c
    heap->cnt = len;

    // 从叶子结点出发向上调整,最后一个叶子结点的双亲结点就是最后一个非叶子结点
    for (int i = len - 1; i > len / 2; i--)
        adjust_max_heap(heap, i);

    return heap;
}

// 销毁堆
void destroy_max_heap(MaxHeap *heap)
{
    free(heap->base);
    free(heap);
}

// 堆是否为空
bool empty_max_heap(MaxHeap *heap)
{
    return 0 == heap->cnt;
}

// 堆是否为满
bool full_max_heap(MaxHeap *heap)
{
    return heap->cal == heap->cnt;
}

// 入堆
bool push_max_heap(MaxHeap *heap, int val)
{
    if (full_max_heap(heap))
        return false;

    // 在末尾添加一个新结点
    heap->base[heap->cnt++] = val;
    // 从最后一个结点出发向上调整
    adjust_max_heap(heap, heap->cnt - 1);
}

// 出堆
bool pop_max_heap(MaxHeap *heap)
{
    if (empty_max_heap(heap))
        return false;

    // 使用最后一个结点覆盖根结点,并且结点的数量减1
    heap->base[0] = heap->base[--heap->cnt];

    // 从根结点出发向下调整
    int parent = 0;
    while (parent < heap->cnt)
    {
        // 假定根结点是根、左右中的最大结点,并计算出它的左右子树的下标
        int max = parent, left = LEFT_CHILD(parent), right = RIGHT_CHILD(parent);

        // 如果左子树下标合法，且左子树的值大于最大结点，则把左子记作最大结点
```

```c
        if (left < heap->cnt && heap->base[max] < heap->base[left])
            max = left;
        // 如果右子树下标合法，且右子树的值大于最大结点，则把右子记作最大结点
        if (right < heap->cnt && heap->base[max] < heap->base[right])
            max = right;

        // 如果最大结点就是根结点，则不需要再向下调整，
        if (max == parent)
            break;

        // 把最大结点与根结点交换，从从大结点处继续向下调整
        swap(heap->base[max], heap->base[parent]);
        parent = max;
    }
    return true;
}

// 堆顶
int top_max_heap(MaxHeap *heap)
{
    if (empty_max_heap(heap))
        return -1;
    return heap->base[0];
}

// 元素数量
size_t size_max_heap(MaxHeap *heap)
{
    return heap->cnt;
}

int main(int argc, const char *argv[])
{
    MaxHeap *heap = create_max_heap(10);
    int arr[10];
    for (int i = 0; i < 10; i++)
    {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
        push_max_heap(heap, arr[i]);
    }
    printf("\n");
    while (!empty_max_heap(heap))
    {
        printf("%d ", top_max_heap(heap));
        pop_max_heap(heap);
    }
    return 0;
}
```

11、赫夫曼树

赫夫曼树介绍：

赫夫曼树是一种带权重的树，它的结点中有一项数据是权重信息，根据结点中的权重生成一棵二叉树，结点的权生越大，离根结点就越近。

赫夫曼树的创建过程：

1、获取值和权重信息、。

2、根据值和权重信息生成若干棵度为0的树，存储到小根堆中形成森林。

3、从小根堆获取到两棵权重最轻的树，把这两棵树的权重相加生成一个新的只有权重的空白树，它的左子树指向权重较小的树，右子树指向权较大的树，然后再把新的树添加到小根堆。

4、重复步骤3，直到小根堆中只有一棵树，它就是创建出的赫夫曼树的根结点。

赫夫曼树的性质：

1、赫夫曼树只有前序遍历才有意义。

2、赫夫曼树的非叶子结点都是只有权重信息，而没有值的空白结点，反之，所有叶子结点都是既有权重又有值的结点。

3、结点的权重越大，结点的高度编号就是越小，离根结点就越近，前遍历访问的速度就越快，反之结点的权重越小，结点的高度编号就越大，离根结点就是越远，前序遍历访问的速度就越慢。

赫夫曼编码：

从赫夫曼树的根结点出发进行前序遍历，遍历过程中把往左子树遍历记作0，把往右子树遍历记作1，到达叶子结点时，走过的路径所形成的01信息被称为赫夫曼编码，使用这种编码就能访问到赫夫曼树根结点出发访问到叶子结点中的数据。

赫夫曼树编码的特点就是权重越大的数据，生成的编码就越短，权重越小的数据，生成的编码就越长。

赫夫曼树编码可以用于提高数据传输效率，还有文件的压缩和解压。

赫夫曼树与赫夫曼编码的应用：

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <stdbool.h>
#include <limits.h>
#include <unistd.h>
#include <math.h>
#include "heap.h"

typedef struct CharCode
{
    uint32_t code; // 赫夫曼编码
    uint8_t cnt;   // 编码位数
} CharCode;

// 赫夫曼编码数组
CharCode code_arr[128];

// 赫夫曼树结点
typedef struct TreeNode
{
```

```c
    uint8_t data;    // 数据
    uint32_t weight; // 权重
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

// 用于比较结点权重的回调函数，给小堆根使用的
int node_cmp(const void *p1, const void *p2)
{
    if (((TreeNode *)p1)->weight > ((TreeNode *)p2)->weight)
        return 1;
    if (((TreeNode *)p1)->weight < ((TreeNode *)p2)->weight)
        return -1;
    return 0;
}

TreeNode *create_node(char data, size_t weight)
{
    TreeNode *node = malloc(sizeof(TreeNode));
    node->data = data;
    node->weight = weight;
    node->left = NULL;
    node->right = NULL;
    return node;
}

void _create_code(TreeNode *root, uint8_t path, uint32_t code, uint8_t cnt)
{
    if (NULL == root)
        return;

    code = (code << 1) + path;

    // 到达叶子结点时，记录赫夫曼编码
    if (0 != root->data)
    {
        code_arr[root->data].code = code;
        code_arr[root->data].cnt = cnt;
        return;
    }

    _create_code(root->left, 0, code, cnt + 1);
    _create_code(root->right, 1, code, cnt + 1);
}

// 生成赫夫曼编码
void create_code(TreeNode *root)
{
    _create_code(root->left, 0, 0, 1);
    _create_code(root->right, 1, 0, 1);
}

// 生成赫夫曼树
TreeNode *init_tree(char *val_arr, size_t *weight_arr, size_t len)
{
    // 创建存储森林的小根堆
    Heap *heap = create_heap(len, node_cmp);
    for (int i = 0; i < len; i++)
```

```
    {
        push_heap(heap, create_node(val_arr[i], weight_arr[i]));
    }

    // 直到堆中只有一个结点，这就是赫夫曼树的根结点
    while (1 < size_heap(heap))
    {
        // 从堆中获取一个权重最小的树,当作左子树
        TreeNode *left = top_heap(heap);
        pop_heap(heap);

        // 从堆中获取一个权重最小的树，当作右子树
        TreeNode *right = top_heap(heap);
        pop_heap(heap);

        // 左右子树的权重相加生成一个新的空白结点
        TreeNode *root = create_node(0, left->weight + right->weight);
        root->left = left;
        root->right = right;

        // 新的结点加入小根堆
        push_heap(heap, root);
    }

    TreeNode *root = top_heap(heap);
    destroy_heap(heap);

    return root;
}

// 压缩文件
int compress_file(const char *oldfile, const char *newfile)
{
    FILE *rfp = fopen(oldfile, "r");
    if (NULL == rfp)
    {
        printf("%s文件打开失败! \n", oldfile);
        return -1;
    }

    FILE *wfp = fopen(newfile, "w");
    if (NULL == wfp)
    {
        printf("压缩文件失败! \n");
        return -1;
    }

    fseek(rfp, 0, SEEK_END);
    fprintf(wfp, "%ld\n", ftell(rfp));
    rewind(rfp);

    uint8_t buf[4096], cnt = 0;
    uint32_t data = 0, offset = 0xffffffff;

    int ret;
    while (ret = fread(buf, 1, sizeof(buf), rfp))
    {
        for (int i = 0; i < ret; i++)
```

```
        {
            if (32 > cnt + code_arr[buf[i]].cnt)
            {
                data = (data << code_arr[buf[i]].cnt) + code_arr[buf[i]].code;
                cnt += code_arr[buf[i]].cnt;
            }
            else
            {
                // 32-cnt 计算出data中还余多少位
                data = (data << (32 - cnt)) + (code_arr[buf[i]].code >> (code_arr[buf[i]].cnt - (32 - cnt)));
                fwrite(&data, 1, sizeof(data), wfp);

                data = (~(offset << (code_arr[buf[i]].cnt - (32 - cnt)))) & code_arr[buf[i]].code;
                cnt = code_arr[buf[i]].cnt - (32 - cnt);
            }
        }
    }

    data = data << (32 - cnt);
    fwrite(&data, 1, sizeof(data), wfp);

    fclose(rfp);
    fclose(wfp);
    return 0;
}

int dp_file(TreeNode *root, const char *filename, const char *oldfile)
{
    FILE *rfp = fopen(filename, "r");
    if (NULL == rfp)
    {
        printf("解压失败! \n");
        return -1;
    }

    long filesize;
    fscanf(rfp, "%ld\n", &filesize);

    FILE *wfp = fopen(oldfile, "w");
    if (NULL == wfp)
    {
        printf("解压失败! \n");
        return -1;
    }

    unsigned int data;
    TreeNode *node = root;
    while (fread(&data, 1, sizeof(data), rfp))
    {
        for (int i = 0; i < 32; i++)
        {
            if ((data << i) & 0x80000000)
                node = node->right;
            else
                node = node->left;
```

```c
            if (0 != node->data)
            {
                fwrite(&node->data, 1, 1, wfp);
                node = root;
            }
        }
    }

    fclose(rfp);
    fclose(wfp);

    return truncate(oldfile, filesize);
}

int main(int argc, const char *argv[])
{
    char val_arr[] = {9, 10, 12, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126};

    size_t weight_arr[] = {16468, 29797, 15, 173045, 243, 916, 8025, 11, 24, 571, 1177, 8000, 8016, 16626, 517, 7384, 1855, 8775, 13269, 5276, 4848, 3603, 2032, 2593, 886, 3292, 805, 1219, 1360, 563, 4162, 943, 1248, 1032, 104, 241, 8184, 2145, 6653, 4971, 12331, 3990, 4196, 4170, 9197, 92, 856, 8929, 5168, 8000, 8562, 7509, 116, 11576, 10356, 12187, 4616, 1346, 2660, 3251, 1510, 859, 213, 808, 195, 16, 48314, 739, 35359, 8675, 21905, 27939, 79328, 25790, 8803, 17961, 48009, 361, 2336, 22424, 11352, 47506, 34893, 13171, 470, 40014, 35301, 57634, 16370, 4914, 6093, 4605, 6933, 2144, 440, 577, 435, 11};

    // 数据+权重构建出森林存储到小根堆里面
    TreeNode *root = init_tree(val_arr, weight_arr, sizeof(val_arr));
    create_code(root);
    printf("%d\n", compress_file("/usr/include/elf.h", "test.dat"));
    printf("%d\n", dp_file(root, "test.dat", "test.h"));
    return 0;
}
```

六、图型结构

1、图的介绍

图是一种比较复杂的数据结构，在线性表中数据元素之间仅有线性关系，每个元素只有一个直接前驱和直接后继（元素之间只存在一对一关系），在树形结构中元素之间有着明显的层次关系，每一层的元素只能和下层的多个元素有关系（元素之间存在一对多关系），而在图形结构中，任意两个结点之间都可能有关系（元素之间存在多对多关系）。

2、图的定义和术语

顶点：图中的数据元素被称为顶点，一般使用V表示图的顶点的有穷非空集合。

弧：两个顶点之间的关系记作<v，w>，表示能从顶点v到达顶点w，也就是v能到w，但w不一定能到v，我们称v为弧尾或初始点，称w为弧头或终端点。

有向图：由弧+顶点构成的图叫有向图，也就是顶点之间是单行道。

边：两个顶点之间的关系记录（v，w），表示既能从v到w，也能从w到v，我们称v和w之间的关系是一条边。

** 无向图：**由边+顶构成的图有无向图，顶点之间的双行道。

注意：一般使用G代表图，V顶点的集合，VR代表弧或边的集合，n代表顶点的数目，e代表边或弧的数目，我们不讨论顶点到自己的边或弧。

完全图：在无向图中，e的取值范围是0 ~ n/2(n-1)，如果无向图的边的数量达到最大值这种无向图称称为完全图。

有向完全图：在有向图中，e的取值范围是0 ~ n(n-1)，如果有向图的弧的数量达到最大值这种有向图称称为有向完全图。

稀疏图和稠密图：如果图中的边和弧很少，e<nlogn 这种图被称为稀疏图，反之称为稠密图。

权和网：如果图中的顶点到另一个顶点需要代价（距离或耗费），那么在表示边或弧的时候需要附加数据，附加的数据就叫做权，带权的图通常被称为网，这也是互联网的由来。

子图：假定有两个图G1和G2，如果G1的顶点集合是G2的顶点集合的子集，且G1的边或弧集合是G2的边或弧集合的子集，则称G1是G2的子图。

邻接点：在无向图中如果有一条边（v，w），则v，w两个顶点互为邻接点，即v和w相邻接，边（v，w）依附于顶点v，w，或者说（v，w）和顶点v、w相关联。

顶点v的度：在无向图中与顶点v相关联的边的数量。

顶点v的入度和出度：在有向图中，以顶点v作为弧头的弧的数量称为顶点的入度，以顶点v作为弧尾的弧的数量称为顶点的出度。

路径：从顶点v到达顶点w所经历的顶点序叫做路径，路径的长度就是边或弧的数目。

回路或环：起始点和终点相同的路径称为回路或环。

简单路径：路径中顶点不重复出现的路径称为简单路径。

简单回路或简单环：起始点和终点相同且其余顶点不重复出现，被称为简单回路或简单环。

连通图：在无向图中，从顶点v到顶点w有路径，则称v和w是连通的，如果图中任意两个顶点都是连通的，则称图为连通图。

连通分量：G1和G2都是连通图，且G1是G2的子图，则称G1是G2的连通分量或极大连通子图。

强连通图：在有向图中，如果任意一对顶点都双向连通，则称图为强连通图。

强连通分量：G1和G2都是强连通图，且G1是G2的子图，则称G1是G2的强连通分量或极大强连通子图。

3、图的存储结构

1、邻接矩阵

使用两个数组来存储数据元素（顶点）和数据元素之间的关系（边或弧）的信息。

一维数组：用于存储顶点。

二维数组：用于存储弧或边。

邻接矩阵的优点：

访问速度快，方便计算结点的度、入度、出度。

邻接矩阵的缺点：

顶点的容量有限，扩容非常麻烦，只适合存储稠密图，存储稀疏图时会有大量的内存浪费。

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// 无向图
typedef struct Grahh
{
    size_t cal;   // 顶点容量
    size_t cnt;   // 顶点数量
    char *vertex; // 存储顶点的一维数组
    char **edge;  // 存储边的二维数组
} Graph;

Graph *create_graph(size_t cal)
{
    Graph *graph = malloc(sizeof(Graph));
    graph->vertex = malloc(sizeof(char) * cal);
    graph->cal = cal;
    graph->cnt = 0;

    graph->edge = malloc(sizeof(char *) * cal);
    for (int i = 0; i < cal; i++)
    {
        graph->edge[i] = calloc(sizeof(char), cal);
    }

    return graph;
}

bool add_vertex_graph(Graph *graph, char vertex)
{
    if (graph->cnt >= graph->cal)
        return false;

    for (int i = 0; i < graph->cnt; i++)
    {
        if (graph->vertex[i] == vertex)
            return false;
    }

    graph->vertex[graph->cnt++] = vertex;
    return true;
}

bool add_edge_graph(Graph *graph, char v1, char v2)
{
    int v1_index = -1, v2_index = -1;
    for (int i = 0; i < graph->cnt; i++)
    {
        if (graph->vertex[i] == v1)
```

```
            v1_index = i;
        if (graph->vertex[i] == v2)
            v2_index = i;
    }

    if (-1 == v1_index || -1 == v2_index || v1_index == v2_index)
        return false;

    graph->edge[v1_index][v2_index] = 1;
    graph->edge[v2_index][v1_index] = 1;
    return true;
}

// 查询顶点在一维数组中的位置，如果不存在则返回-1
int query_vertex_graph(Graph *graph, char vertex)
{
    for (int i = 0; i < graph->cnt; i++)
    {
        if (vertex == graph->vertex[i])
            return i;
    }
    return -1;
}

// 查询顶点第一个邻接点，如果不存在则返回'\0'
char first_vertex_graph(Graph *graph, char vertex)
{
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return '\0';

    for (int i = 0; i < graph->cnt; i++)
    {
        if (1 == graph->edge[index][i])
            return graph->vertex[i];
    }

    return '\0';
}

// 删除顶点
bool del_vertex_graph(Graph *graph, char vertex)
{
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return false;

    for (int i = 0; i < graph->cnt; i++)
    {
        graph->edge[index][i] = 0;
        graph->edge[i][index] = 0;
    }

    return true;
}

void _dfs_graph(Graph *graph, int index, bool *flags)
{
```

```c
    if (!flags[index])
        return;

    printf("%c\n", graph->vertex[index]);
    flags[index] = false;

    for (int i = 0; i < graph->cnt; i++)
    {
        if (1 == graph->edge[index][i])
            _dfs_graph(graph, i, flags);
    }
}

// 深度优先遍历，与树的前序遍历类似
void dfs_graph(Graph *graph)
{
    bool flags[graph->cnt];
    for (int i = 0; i < graph->cnt; i++)
        flags[i] = true;

    for (int i = 0; i < graph->cnt; i++)
        _dfs_graph(graph, i, flags);
}

// 广度优先遍历，与树的层序遍历一样，需要队列结构配合
void bfs_graph(Graph *graph)
{
    bool flags[graph->cnt];
    for (int i = 0; i < graph->cnt; i++)
        flags[i] = true;

    int queue[graph->cnt], front = 0, rear = 0;

    for (int i = 0; i < graph->cnt; i++)
    {
        if (flags[i])
        {
            flags[i] = false;
            queue[rear++] = i;
        }

        while (front != rear)
        {
            int index = queue[front++];
            printf("%c ", graph->vertex[index]);

            for (int j = 0; j < graph->cnt; j++)
            {
                if (1 == graph->edge[index][j] && flags[j])
                {
                    flags[j] = false;
                    queue[rear++] = j;
                }
            }
        }
    }
}

```

```c
// 销毁图
void destroy_graph(Graph *graph)
{
    for (int i = 0; i < graph->cnt; i++)
    {
        free(graph->edge[i]);
    }
    free(graph->edge);
    free(graph->vertex);
    free(graph);
}

int main(int argc, const char *argv[])
{
    Graph *graph = create_graph(5);
    add_vertex_graph(graph, 'A');
    add_vertex_graph(graph, 'B');
    add_vertex_graph(graph, 'C');
    add_vertex_graph(graph, 'D');
    add_vertex_graph(graph, 'E');

    add_edge_graph(graph, 'A', 'C');
    add_edge_graph(graph, 'A', 'D');
    add_edge_graph(graph, 'C', 'B');
    add_edge_graph(graph, 'B', 'D');
    // del_vertex_graph(graph,'C');
    printf("------------------------\n");
    for (int i = 0; i < graph->cnt; i++)
    {
        for (int j = 0; j < graph->cnt; j++)
        {
            printf("%hhd ", graph->edge[i][j]);
        }
        printf("\n");
    }
    // destroy_graph(graph);
    // printf("%c\n",first_vertex_graph(graph,'D'));
    // dfs_graph(graph);
    bfs_graph(graph);
    return 0;
}
```

2、邻接表

采用一维数组加单向链表的方式存储存储。

一维数组：用于存储顶点和单向链表的头指针。

单向链表：存储着从该顶点出发的弧或边。

邻接表优点：

与邻接矩阵相比，它可以节约内存，有多少条边或弧就创建多少个链表节点，比较适合存储稀疏图。

邻接表缺点：

计算入度、删除节点时比较麻烦。

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// 弧
typedef struct Bow
{
    int dest;
    struct Bow *next;
} Bow;

// 顶点
typedef struct Vertex
{
    char data;
    Bow first; // 单链表的空白头节点
} Vertex;

// 有向图
typedef struct Graph
{
    Vertex *vertex;
    size_t cal;
    size_t cnt;
} Graph;

// 创建图
Graph *create_graph(size_t cal)
{
    Graph *graph = malloc(sizeof(Graph));
    graph->vertex = malloc(sizeof(Vertex) * cal);
    graph->cal = cal;
    graph->cnt = 0;
    return graph;
}

// 添加顶点
bool add_vertex_graph(Graph *graph, char vertex)
{
    if (graph->cnt >= graph->cal)
        return false;

    // 顶点不能重复
    for (int i = 0; i < graph->cnt; i++)
        if (vertex == graph->vertex[i].data)
            return false;

    graph->vertex[graph->cnt].data = vertex;
    graph->vertex[graph->cnt++].first.next = NULL;
    return true;
}

// 查询顶点
```

```c
int query_vertex_graph(Graph *graph, char vertex)
{
    for (int i = 0; i < graph->cnt; i++)
    {
        if (graph->vertex[i].data == vertex)
            return i;
    }

    return -1;
}

// 添加弧
bool add_bow_graph(Graph *graph, char bow_tail, char bow_head)
{
    int index_tail = query_vertex_graph(graph, bow_tail);
    if (-1 == index_tail)
        return false;

    int index_head = query_vertex_graph(graph, bow_head);
    if (-1 == index_head)
        return false;

    Bow *bow = malloc(sizeof(Bow));
    bow->dest = index_head;

    bow->next = graph->vertex[index_tail].first.next;
    graph->vertex[index_tail].first.next = bow;
    return true;
}

// 删除顶点
bool del_vertex_graph(Graph *graph, char vertex)
{
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return false;

    if (NULL != graph->vertex[index].first.next)
    {
        Bow *bow = graph->vertex[index].first.next;
        graph->vertex[index].first.next = bow->next;
        free(bow);
    }
    graph->vertex[index].data = '\0';

    for (int i = 0; i < graph->cnt; i++)
    {
        for (Bow *bow = &graph->vertex[i].first; NULL != bow->next; bow = bow->next)
        {
            if (index == bow->next->dest)
            {
                Bow *tmp = bow->next;
                bow->next = tmp->next;
                free(tmp);
            }
        }
    }
```

```c
    return true;
}

// 计算入度，计算出有多个弧的弧头是某顶点
int indegree_vertex_graph(Graph *graph, char vertex)
{
    // 打到顶点，遍历所有弧，dest等于vertex下的弧有多少条，如果找不到则返回-1
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return -1;

    int indegree = 0;
    for (int i = 0; i < graph->cnt; i++)
    {
        Bow *bow = graph->vertex[i].first.next;
        while (NULL != bow)
        {
            if (index == bow->dest)
                indegree++;
            bow = bow->next;
        }
    }

    return indegree;
}

// 计算出度
int outdegree_vertex_graph(Graph *graph, char vertex)
{
    // 找到顶点，从它出发的弧有多少条，如果找不到则返回-1
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return -1;

    int outdegree = 0;
    Bow *bow = graph->vertex[index].first.next;
    while (NULL != bow)
    {
        outdegree++;
        bow = bow->next;
    }

    return outdegree;
}

// 深度优先遍历
void _dfs_graph(Graph *graph, int index, bool *flags)
{
    flags[index] = false;
    printf("%c ", graph->vertex[index].data);

    Bow *bow = graph->vertex[index].first.next;
    while (NULL != bow)
    {
        if (flags[bow->dest])
            _dfs_graph(graph, bow->dest, flags);

        bow = bow->next;
```

```c
    }
}

void dfs_graph(Graph *graph)
{
    // 参考昨天邻接矩阵的业务逻辑
    bool flags[graph->cnt];
    for (int i = 0; i < graph->cnt; i++)
        flags[i] = true;

    for (int i = 0; i < graph->cnt; i++)
        if (flags[i])
            _dfs_graph(graph, i, flags);
    printf("\n");
}

// 广度优先遍历
void bfs_graph(Graph *graph)
{
    bool flags[graph->cnt];
    for (int i = 0; i < graph->cnt; i++)
        flags[i] = true;

    int queue[graph->cnt], front = 0, rear = 0;

    for (int i = 0; i < graph->cnt; i++)
    {
        if (flags[i])
        {
            flags[i] = false;
            queue[rear++] = i;
        }

        while (front != rear)
        {
            int index = queue[front++];
            printf("%c ", graph->vertex[index].data);

            Bow *bow = graph->vertex[index].first.next;
            while (NULL != bow)
            {
                if (flags[bow->dest])
                {
                    flags[bow->dest] = false;
                    queue[rear++] = bow->dest;
                }
                bow = bow->next;
            }
        }
    }
    printf("\n");
}

// 销毁图
void destroy_graph(Graph *graph)
{
    for (int i = 0; i < graph->cnt; i++)
    {
```

```c
        Bow *bow = &graph->vertex[i].first;
        while (NULL != bow->next)
        {
            Bow *tmp = bow->next;
            bow->next = tmp->next;
            free(tmp);
        }
    }
    free(graph->vertex);
    free(graph);
}

int main(int argc, const char *argv[])
{
    Graph *graph = create_graph(10);
    add_vertex_graph(graph, 'A');
    add_vertex_graph(graph, 'B');
    add_vertex_graph(graph, 'C');
    add_vertex_graph(graph, 'D');
    add_vertex_graph(graph, 'E');

    add_bow_graph(graph, 'A', 'D');
    add_bow_graph(graph, 'A', 'E');
    add_bow_graph(graph, 'A', 'C');
    add_bow_graph(graph, 'C', 'B');
    add_bow_graph(graph, 'B', 'D');
    // add_bow_graph(graph,'E','D');
    /*
    del_vertex_graph(graph,'E');
    for(int i=0; i<graph->cnt; i++)
    {
        printf("%d %c first->%p",i,graph->vertex[i].data,graph->vertex[i].first.next);
        for(Bow* bow=graph->vertex[i].first.next; NULL!=bow; bow=bow->next)
        {
            printf(" %p dest:%d ",bow,bow->dest);
        }
        printf("\n");
    }
    printf("%d\n",outdegree_vertex_graph(graph,'B'));
    printf("%d\n",indegree_vertex_graph(graph,'A'));
    */
    dfs_graph(graph);
    bfs_graph(graph);
    destroy_graph(graph);
    return 0;
}
```

3、十字链表

采用一维数组加两个十字链表进行存储，之所以叫十字链表，是因为它的链表弧节点会被两个链表指向，同时弧节点有两个指针，指向后续的弧节点。

一维数组：存储了顶点数据，两个链表头指针。

出度链表：指向的是以该结点作为弧尾的弧节点。

入度链表：指向的是以该结点作为弧头的弧节点。

十字链表的优点：

与邻接矩阵相比，它更节约内存，与邻接表相比它更方便计算顶点的出度。

十字链表的缺点：

删除顶点和删除弧时比较麻烦，也可以表示与实现无向图，但更合适表示与实现有向图。

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// 弧
typedef struct Bow
{
    int tailvex;        // 弧尾的顶点下标
    int headvex;        // 弧头的顶点下标
    struct Bow *tlink; // 指向相同弧尾的弧节点
    struct Bow *hlink; // 指向相同弧头的弧节点
} Bow;

// 顶点
typedef struct Vertex
{
    char data;
    Bow *inlink;
    Bow *outlink;
} Vertex;

// 图
typedef struct Graph
{
    Vertex *vertex;
    size_t cal;
    size_t cnt;
} Graph;

// 创建图
Graph *create_graph(size_t cal)
{
    Graph *graph = malloc(sizeof(Graph));
    graph->vertex = malloc(sizeof(Vertex) * cal);
    graph->cal = cal;
    graph->cnt = 0;
    return graph;
}
// 查询顶点下标
int query_vertex_graph(Graph *graph, char vertex)
{
    for (int i = 0; i < graph->cnt; i++)
        if (vertex == graph->vertex[i].data)
            return i;

    return -1;
}

// 添加顶点
bool add_vertex_graph(Graph *graph, char vertex)
```

```c
{
    // 如果顶点已经满，或顶点已经存在则返回false
    if (graph->cnt >= graph->cal || -1 != query_vertex_graph(graph, vertex))
        return false;

    graph->vertex[graph->cnt].data = vertex;
    graph->vertex[graph->cnt].inlink = NULL;
    graph->vertex[graph->cnt++].outlink = NULL;
    return true;
}

// 添加弧
bool add_bow_graph(Graph *graph, char tailvex, char headvex)
{
    // 如果弧头或弧尾的顶点不存在则添加失败
    int tailindex = query_vertex_graph(graph, tailvex);
    int headindex = query_vertex_graph(graph, headvex);
    if (-1 == tailindex || -1 == headindex)
        return false;

    // 创建一个弧
    Bow *bow = malloc(sizeof(Bow));
    bow->tailvex = tailindex;
    bow->headvex = headindex;

    bow->tlink = graph->vertex[tailindex].outlink;
    graph->vertex[tailindex].outlink = bow;

    bow->hlink = graph->vertex[headindex].inlink;
    graph->vertex[headindex].inlink = bow;

    return true;
}

// 删除顶点
bool del_vertex_graph(Graph *graph, char vertex)
{
    // 找到顶点的下标，如果找不到则返回false
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return false;

    // 以它作为弧尾的弧要全部断开
    graph->vertex[index].inlink = NULL;

    // 以它作为弧头的弧要全部删除
    for (int i = 0; i < graph->cnt; i++)
    {
        Bow *out = graph->vertex[i].outlink;
        while (NULL != out && NULL != out->tlink)
        {
            if (out->tlink->headvex == index)
            {
                Bow *tmp = out->tlink;
                out->tlink = tmp->tlink;
                free(tmp);
            }
```

```
            out = out->tlink;
        }
        if (NULL != graph->vertex[i].outlink &&
            index == graph->vertex[i].outlink->headvex)
        {
            Bow *tmp = graph->vertex[i].outlink;
            graph->vertex[i].outlink = tmp->tlink;
            free(tmp);
        }
    }
    return true;
}

// 删除弧
bool del_bow_graph(Graph *graph, char tailvex, char headvex)
{
    // 找到弧头、弧尾顶点的下标，如果有一个找不到就返回false
    int tailindex = query_vertex_graph(graph, tailvex);
    int headindex = query_vertex_graph(graph, headvex);
    if (-1 == tailindex || -1 == headindex)
        return false;

    // 删除弧，并且要保证两个链表不断开
    Bow *out = graph->vertex[tailindex].outlink;
    while (NULL != out && NULL != out->tlink)
    {
        if (out->tlink->headvex == headindex)
        {
            out->tlink = out->tlink->tlink;
            break;
        }
        out = out->tlink;
    }
    if (NULL != graph->vertex[tailindex].outlink &&
        headindex == graph->vertex[tailindex].outlink->headvex)
    {
        graph->vertex[tailindex].outlink = graph->vertex[tailindex].outlink->tlink;
    }

    Bow *in = graph->vertex[headindex].inlink;
    while (NULL != in && NULL != in->hlink)
    {
        if (in->hlink->tailvex == tailindex)
        {
            Bow *tmp = in->hlink;
            in->hlink = tmp->hlink;
            free(tmp);
            break;
        }
        in = in->hlink;
    }

    if (NULL != graph->vertex[headindex].outlink &&
        tailindex == graph->vertex[headindex].outlink->tailvex)
    {
        Bow *tmp = graph->vertex[headindex].inlink;
        graph->vertex[headindex].inlink = tmp->hlink;
```

```
        free(tmp);
    }
    return true;
}

// 计算入度
int indegree_vertex_graph(Graph *graph, char vertex)
{
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return -1;

    int indegree = 0;
    Bow *in = graph->vertex[index].inlink;
    while (NULL != in)
    {
        indegree++;
        in = in->hlink;
    }

    return indegree;
}
// 计算出度
int outdegree_vertex_graph(Graph *graph, char vertex)
{
    int index = query_vertex_graph(graph, vertex);
    if (-1 == index)
        return -1;

    int outdegree = 0;
    Bow *out = graph->vertex[index].outlink;
    while (NULL != out)
    {
        outdegree++;
        out = out->tlink;
    }
    return outdegree;
}

// 深度优先
void _dfs_graph(Graph *graph, int index, bool *flags)
{
    printf("%c ", graph->vertex[index].data);
    flags[index] = false;

    Bow *out = graph->vertex[index].outlink;
    while (NULL != out)
    {
        if (flags[out->headvex])
            _dfs_graph(graph, out->headvex, flags);
        out = out->tlink;
    }
}

void dfs_graph(Graph *graph)
{
    bool flags[graph->cnt];
    for (int i = 0; i < graph->cnt; i++)
```

```c
        flags[i] = true;

    for (int i = 0; i < graph->cnt; i++)
        if (flags[i])
            _dfs_graph(graph, i, flags);
    printf("\n");
}

// 广度优先
void bfs_graph(Graph *graph)
{
    bool flags[graph->cnt];
    for (int i = 0; i < graph->cnt; i++)
        flags[i] = true;

    int queue[graph->cnt], front = 0, rear = 0;
    for (int i = 0; i < graph->cnt; i++)
    {
        if (flags[i])
        {
            queue[rear++] = i;
            flags[i] = false;
        }
        while (front != rear)
        {
            int index = queue[front++];
            printf("%c ", graph->vertex[index].data);

            Bow *out = graph->vertex[index].outlink;
            while (NULL != out)
            {
                if (flags[out->headvex])
                {
                    queue[rear++] = out->headvex;
                    flags[out->headvex] = false;
                }
                out = out->tlink;
            }
        }
    }
    printf("\n");
}

// 销毁图
void destroy_graph(Graph *graph)
{
    for (int i = 0; i < graph->cnt; i++)
        del_vertex_graph(graph, graph->vertex[i].data);

    free(graph->vertex);
    free(graph);
}

int main(int argc, const char *argv[])
{
    Graph *graph = create_graph(10);
    add_vertex_graph(graph, 'A');
    add_vertex_graph(graph, 'B');
```

```
    add_vertex_graph(graph, 'C');
    add_vertex_graph(graph, 'D');
    add_vertex_graph(graph, 'E');
    add_bow_graph(graph, 'A', 'D');
    add_bow_graph(graph, 'A', 'E');
    add_bow_graph(graph, 'A', 'C');
    add_bow_graph(graph, 'C', 'B');
    add_bow_graph(graph, 'B', 'D');
    add_bow_graph(graph, 'E', 'D');
    dfs_graph(graph);
    bfs_graph(graph);
    destroy_graph(graph);
    // del_vertex_graph(graph,'D');
    // del_bow_graph(graph,'A','D');
    /*
    printf("%d\n",indegree_vertex_graph(graph,'B'));
    for(int i=0; i<graph->cnt; i++)
    {
        printf("vex:%c",graph->vertex[i].data);
        printf("\n\t");
        Bow* out = graph->vertex[i].outlink;
        while(NULL != out)
        {
            printf("%d->%d ",out->tailvex,out->headvex);
            out = out->tlink;
        }
        printf("\n\t");
        Bow* in= graph->vertex[i].inlink;
        while(NULL != in)
        {
            printf("%d<-%d ",in->headvex,in->tailvex);
            in = in->hlink;
        }
        printf("\n");
    }

    */
    return 0;
}
```

4、邻接多重表：

是对邻接表一种扩展，在邻接表边的基础上，增加了访问标记、组成边的两个顶点的下标、指向依附这两个顶点的下一条边的指针。

使用邻接表时，它的一条边的两个节点，分别在第i和第j的两个链表中，这给图的操作带来不便，而邻接多重表的链表节点增加各项成员，方便对图的操作。

七、算法和算法分析

1、什么是算法

对特定问题的求解步骤的一种描述，它是指令的有限序列，其中每一条指令表示一个或多个操作。

2、算法的五个重要特性

- 有穷性
- 确定性
- 可行性
- 输入
- 输出

3、算法设计的要求

- 正确性
- 可读性
- 健壮性
- 高效率与低存储

4、算法效率的度量

算法的执行时间需要依据该算法所编制的程序在计算机上运行所消耗的时间来度量，一般有两种方法：

事后统计的方法：

很多计算机内部都有计时功能，不同的算法的程序可通过一组或若干相同的统计数据以分辨优劣，例如：time ./a.out，但这种方法有两个缺陷，一是必须先运行依据算法编程的程序，如果程序不满足要求编写程序所耗费的人力物力就浪费了，二是所得时间的统计量依赖于计算机的硬件、软件等环境因素，有时容易掩盖算法本身的优劣，因此人们常常采用另一种事前分析法。

事前分析估算的方法：

一个用户高级程序语言编写的程序在计算机上运行所消耗的时间取决于下列因素：

1、依据算法先用何种策略，比如循环操作，可使用函数递归或for、while、do while等循环语句。

2、问题的规模，少量数据的问题求解、大规模数量的问题求解、海量数据的问题求解。

3、书写程序的语言，对于同一种算法，实现的语言级别越高，执行效率就越低。

4、编译程序所产生的机器代码的质量。

5、机器执行指令的速度，相同的编程语言，使用的编译器不同，产生的机器指令就会不同，从而执行的速度就会不同。

总结：同一个算法用不同的语言实现，或者用不同的编译器进行编译，或者不同的计算机上运行，效率均不相同，这表明使用绝对的时间单位衡量算法的效率是不合适的，抛开这些与计算机硬件、软件有关因素，可以认为特定算法“运行工作量”的大小，只依赖于问题的规模，或者说，它是问题规模的函数。

一个算法是由控制结构（顺序、分支、循环等3种基本结构）和原操作（指固有数据类型类型的操作）构成的，则算法时间取决于两者的统合效果，为了便于比较同一问题的不同算法，同学做法是，从算法中选取一种对于 所研究的问题来说是基本操作的原操作，以该基本操作重复执行的次数作为算法的时间量度。

5、时间复杂度

所以：算法中基本操作重复执行的次数是问题规模n的某个函数f(n)，算法的时间量度记叙T（n）=O（f（n）），它表示问题规模n的增大，算法执行时间的增长率和f（n）的增长率相同，称做算法的渐近时间复杂度，简称时间复杂度。

1、常见的时间复杂度有

- O(1)
- O(log2N)/O(logN)
- O(N)
- O(NlogN)
- O(N^2)
- O(2^N)

  注意：一般时间复杂度取的是一种大约情况不会精确到特别详细，也就是不会百分百正确。

2、最差时间复杂度

数据的极端排序顺序会让算法执行效率最差的一种情况。

3、最优时间复杂度

数据的极端排序顺序会让算法执行效率最优的一种情况。

4、平均时间复杂度

数据的排序顺序队伍事物发展规律，算法执行效率接近真实的一种情况。

5、空间复杂度

它代表随着问题规模n的某个函数f(n)，它表示问题规模n的增大，算法所需要的存储空间增长率，被称为算法的空间复杂度。

八、查找算法

1、什么是查找算法

在一个数据序列中，查找某个数据是否存在或存在的位置，在实际开发过程中使用的频率非常高，例如对数据常见的操作有增、删、改、查，增加数据时需要查询新增加的数据是否重复，删除数据时需要先查询到数据所在位置再删除，修改数据时也需要先查询到被修改的数据在什么位置，查找算法在编程中重要性排列在第一位。

2、顺序查找

顺序表的顺序查找：

```
// 在顺序表中按照从前到后的顺序查找数据，如果找到则返回合法的下标，找不到则返回 -1
int linear_search(int* arr,size_t len,int key)
{
    for(int i=0; i<len; i++)
    {
        if(arr[i] == key)
            return i;
    }
    return -1;
}
```

链表的顺序查找：

```
// 在顺序表中按照从前到后的顺序查找数据，如果找到则返回所在节点的地址，找不到则返回NULL
Node* linear_search(Node* head,int key)
{
    for(Node* n=head; NULL!=n; n=n->next)
    {
        if(n->data == key)
            return n;
    }
    return NULL;
}
```

顺序查找的优点：

对数据的有序性没有要求，无论数据是否有序都可以查找。

对数据结构没有要求，无论是顺序表还中链式表都可以查找。

顺序查找的缺点：

查找速度相比其它查找算法慢，最优时间复杂度：O(1)，最差时间复试度：O(N)，平均时间复杂度：O(N)

3、二分查找

数据序列必须有序，然后关键字key与中间数据比较，如果相等则立即返回，如果key小于中间数据，则只需要在中间数据左边继续查找即可，如果key大于中间数据，则只需要在中间数据右边继续查找即可，重复该步骤，直到找到关键字，或中间数据左右两边为空，则查找失败。

```
// 循环语句实现
int binary_search(int* arr, size_t len, int key)
{
    int left = 0 , right = len;
    while(left < right)
    {
        int p = (left + right) / 2;
        if(arr[p] == key)
            return p;
        if(arr[p] > key)
            r = p;
        if(arr[p] < key)
            l = p+1;
    }
    return -1
}

// 函数递归实现
int _binary_search(int* arr, int left, int right, int key)
{
    if(left >= right)
        return -1;
    int p = (left + right) / 2;
    if(arr[p] > key)
        return _binary_search(arr,left,p,key);
    if(arr[p] < key)
        return _binary_search(arr,p+1,right,key);
    return p;
```

```
}

int binary_search(int* arr, size_t len, int key)
{
    return _binary_search(arr,0,len,key);
}
```

二分查找的优点：

查询速度快，时间复杂度：O(log2N)。

二分查找的缺点：

1、对数据的有序性有要求，必须要先排序完成。

2、对数据结构有要求，不适合链式表使用，因为无法计算出中间位置的数据。

4、索引查找

索引查找是在索引表和主表(即线性表的索引存储结构)上进行的查找，但需要先建立索引表，索引表就类似图书的目录，能大提高查找效率。

给顺序表创建索引表：

```
typedef struct Student
{
    int id;
    char name[20];
    char sex;
    short age;
    float src;
}Student;

Student stu[1000];

typedef struct Index
{
    int id;
    void* addr;
}Index;

Index* index[1000];
void create_index(Student* stu,Index* index,size_t len)
{
    for(int i=0; i<len; i++)
    {
        index[i].id = stu[i].id;
        index[i].addr = stu+i;
    }
}
```

索引表的顺序查找：

```c
// 它比直接查询原感受的跳转速度快，如果数据在内存中可以减少跳转的内存页数量，如果数据在机械硬盘中可以减少硬盘磁头的移动距离，如果使用stu[i]，i每增加一下，需要跳转sizeof(Student)个字节，如果使用index[i]，i每增加一下，只需要跳转sizeof(Index)个字节，。
int linear_index(Index* index,size_t len,int id)
{
    for(int i=0; i<len; i++)
    {
        if(index[i].id == id)
            return i;
    }
}
```

索引表二分查找:

```c
// 如果对原表直接排序，则每次需要交换sizeof(Student)，如果对索引表进行排序每次只需要交换sizeof(Index)个字节，数据的交换量对排序算法的程度影响非常大。
void sort_index(Index* index,size_t len)
{
    for(int i=0; i<len-1; i++)
    {
        int min = i;
        for(int j=i+1; j<len; j++)
        {
            if(index[min].id > index[j].id)
                min = j;
        }

        if(i != min)
        {
            Index tmp = index[min];
            index[min] = index[i];
            index[i] = tmp;
        }
    }
}

Student* binary_search(Index* index,size_t len,int id)
{
    int left = 0, right = len;
    while(left < right)
    {
        int p = (left + right) / 2;
        if(index[p].id == id)
            return index[p].ptr;
        if(index[p].id > id)
            right = p;
        if(index[p].id < id)
            left = p+1;
    }
    return NULL;
}
```

给链表创建索引表：

```
// 给链表创建了索引表后，再对索引表进行排序，就可以对链表进行二分查找的效果
void create_index(Node* head,Index* index)
{
    int i = 0;
    for(Node* n=head; NULL!=n; n=n->next)
    {
        index[i].id = ((Student*)n->ptr)->id;
        index[i].addr = n->ptr;
    }
}
```

索引查找的优点：

1、对于顺序表的顺序查找，索引表可以缩小数据的查找范围。

2、对于顺序表的二分查找，索引表可以提高排序速度。

3、对于链表的可以在创建索引表后，进行二分查找。

索引查找的缺点：

使用了额外的存储空间创建索引表，是一种典型的用空间换取时间的策略。

索引查找的使用场景：

针对内存中的顺序表数据，索引查找提升的速度并不是特别大，而是对存储在机械硬盘中的数据的查找速度提升非常大，因此在数据库中索引查找使用的特别多。

5、分块查找

先对数据进行分块，可以根据日期进行分块，可以对整形数据根据数据的最后一位分为10个分表，针对字符串数据可以根据第一个字母分为26个分表，然后再对这些分表进行二分查找、顺序查找，它是分治思想的具体实现。

分块查找的优点：

可以把海量数据进行分化，降低数据规模，从而提升查找速度。

分块查找的缺点：

操作比较麻烦，可能会有一定的内存浪费。

6、二叉排序树和平衡二叉树

二叉排序树也叫二叉搜索树是根据数据的值进行构建的，然后进行前序遍历查找，它的查找原理还是二分查找，我在二叉搜索树中已经详细过，在此不再赘述。

平衡二叉树首先是一棵二叉排序树或二叉搜索树，二叉排序树在创建时，由于数据基本有序，会导致创建出的二叉排序树呈单枝状，或者随机数据的插入、删除导致二叉排序树左右失衡呈单枝状，就会导致二叉树排序的查询速度接近单向链表的O(N)；

平衡二叉树要求左右子树的高度差不超过1，这样就会让二叉排序树左右均匀，让二叉排序树的查找速度接近二分查找（要了解AVL树和红黑树的区别）。

7、哈希表查找

在查找数据时需要进行一系列的比较运算，无论是顺序查找、二分查找、索引查找、这类的查找算法都是建立在比较的基础上的，但最理想的查找算法是不经过任何比较，一次存取就能查找到数据，那么就需要在存储位置和它的关键字之间建立一个确定的对应关系，使每个关键字和数据中的唯一的存储位置相对应。在查找时，根据对应关系找到给定的关键字所对应的数据，这样可以不经过比较可以直接取得所查找的数据。

我们称存储位置在关键字之间的对应关系为哈希函数，按这个思想建立的表为哈希表。

问题：假定有100万个0~255之间的随机数，然后输入一个0~255之间的整数，查询出该数一共出现了多少次。

```c
#include <stdio.h>
#include <stdlib.h>

int arr[1000000];
int cnts[256];
// 最简单的哈希表
int main(int argc,const char* argv[])
{
    for(int i=0; i<1000000; i++)
    {
        arr[i] = rand() % 256;
    }

    // 把数据会值作为数组cnts的下标(把要关键字与存储位置相对应)，直接定值法，H(key)=key
    for(int i=0; i<1000000; i++)
    {
        cnts[arr[i]]++;
    }

    unsigned char num;
    printf("请输入一个整数(0~255)：");
    scanf("%hhu",&num);

    printf("%d\n",cnts[num]);
    return 0;
}
```

以上例子有很大的局限性，很多时候无法使用数据的值作为数组的下标，因为可能会出现数据量少，但差值范围太大，会导致哈希表过大，实用性不强，所以就需要对关键字的值进行处理，处理方法就是哈希函数，进而缩小哈希表，然而设计哈希函数计算出的结果可能会出现重复，因此设计出合适的哈希函数、解决冲突问题是哈希表和哈希查找的关键，而且很多时候无法设计出哈希函数，比如浮点型数据，汉字字符串等很难设计出合适的哈希函数。

设计哈函数的方法：

直接定值法：

取关键字的值或某个线性函数作为k哈希地址：H(key) = key 或H(key)=a*key-b，但这种方法对数据有很高的要求。

数字分析法：

假设以关键字key为基数，并且哈希表中可能出现的数字全部知道，取关键字的某几位为哈希地址，例如学号：

1~2系、3~4为专业、5~6为年级、7~9为入学序号，因此H(key)=key%1000。

以上两种方法局限比较大，但计算出的哈希地址没有冲突的可能，以下方法：平方取中法、折叠法、除留余数法等方法对关键字的要求不要，但计算出的哈希地址可能会有冲突。

解决哈希值冲突的方法：

- 开方地址法
- 再哈希法
- 链地址法
- 创建公共溢出区

哈希查找的优点：

查找速度极快，时间复杂度能达到O(1)。

哈希查找的缺点：

1、局限性比较大，对数据的要求比较高。

2、设计哈希函数麻烦，没具体的设计哈希函数的方法，只是有一些大致的思路。

3、需要建立哈希表，占用了额外的空间。

九、排序算法

1、排序算法介绍

排序是计算机程序设计过程中的重要操作，它的功能是将一个数据元素的任意序列，重新排列成一个按关键字有序的序列。

它之所以重要是因为查找操作非常重要，而有序的顺序表可以采用效率较高二分查找（时间复杂度为O(logn)），而无序的顺序表只能进行顺序查找（时间复杂度为O(N)），而创建二叉排序树、有序二叉树、堆的过程本身就是一个排序过程。

2、排序算法的稳定性

待排序的数据序列中可能存储两个或两个以上的关键字相等的数据，假设ki==kj且 i!=j，如果排序前i<j，排序后也肯定i<j，这种排序算法是稳定的排序，反之是不稳定的排序。

3、十大经典排序算法

经典排序：

能够完成排序操作，但没有任何优化，数据的比较次数一次没少，数据的交换次数一次没少，没有任何优点，不能称得上是算法。

```
// 时间复杂度：O(N^2)，空间复杂度：O(1)，稳定性：稳定
void classic_sort(int* arr,size_t len)
{
    for(int i=0; i<len-1; i++)
    {
        for(int j=i+1; j<len; j++)
        {
            if(arr[i] > arr[j])
                swap(arr[i],arr[j]);
        }
    }
}
```

1、冒泡排序

排序过程像是水中的气泡上升，越大的气泡升的越快，所以叫冒泡排序，从第一个数据开始，让前后相邻的数据进行比较，如果Ki>Ki+1则交换它们，每一趟排序完成一个数据，反复这个过程，直接待排序的数据为1，则结束排序。

和经典排序相比，冒泡排序对数据的有序性敏感，如果一趟排序没有发生交换，则说明所前面的数据都比它的后继数据要小，则可以立即停止，提前结束排序。

注意：如果待排序的数据基本有序，则使用冒泡排序速度最快，因为它数据的有序性敏感可以提前结束。

```
// 最优时间复杂度：O(N)，最差时间复杂度：O(N^2)，平均时间复杂度：O(N^2)，空间复杂度：O(1)，稳定性：稳定
void bubble_sort(int* arr,size_t len)
{
    bool flag = true;
    for(int i=len-1; flag&&i>0; i--)
    {
        flag = false;
        for(int j=0; j<i; j++)
        {
            if(arr[j] > arr[j+1])
            {
                swap(arr[j],arr[j+1]);
                flag = true;
            }
        }
    }
}
```

2、选择排序

从第i个待排序的数据开始，假定它是最小值，并用min记录它的下标，然后与用arr[min]与它后面的数据进行比较，如果有它还小的数据，则更新min的值，排序一趟后，如果min==i，则说明第一个数据是最小值，不需要交换，如果min!=i则交接第一个数据与arr[min]，然后i++，重复这个步骤直到待排序的数据为1（i=len-1），排序结束。

和经典排序相比，它的数据比较次数没有减少，但是它的数据交换数次大大降低(O(N-1))，节约了很多数据交换的时间，虽然时间复杂度没有变化，排序效率比经典排序提高了很多。

注意：选择排序最突出的特点是数据的交换次数最少，如果待排序的数据字节数较多，如：结构、类对象，则使用选择排序速度最快。

```c
// 时间复杂度：O(N^2)，空间复杂度：O(1)，稳定性：不稳定
void select_sort(int* arr,size_t len)
{
    for(int i=0; i<len-1; i++)
    {
        int min = i;
        for(int j=i+1; j<len; j++)
        {
            if(arr[min] > arr[j])
                min = j;
        }
        if(min != i)
            swap(arr[min],arr[i]);
    }
}
```

3、插入排序

往有序的序列中添加新的数据，使序列继续保持有序，具体步骤是：假定新的数据存放在i=len位置，val与它前面的数据逐一进行比较val<arr[i-1]，如果前面的数据大于tmp大，则把前面数据的向后拷贝一下，然后i自减1重复以比较，直到arr[i-1]<=tmp或0==i，则位置i就是tmp应该存放的位置，新插入的数据就排序完成。

可以使用以上方法对无序序列进行排序，把序列看作两个总分，已经有序的部分（数量为1）和待插入的部分（len-1），把待插入部分的数据逐个向前面有序部分插入，就完成了对无序序列进行排序。

注意：顾名思义，插入排序算法适合往有序的序列中添加新的数据，优点是排序过程中没有进行数据交换，节约了大量时间。

```c
// 时间复杂度：O(N^2)，空间复杂度：O(1)，稳定性：稳定
void _insert_sort(int* arr,size_t len,int val)
{
    int i = len;
    while(i>0 && arr[i-1]>val)
    {
        arr[i] = arr[i-1];
        i--;
    }
    arr[i] = val;
}
void insert_sort(int* arr,size_t len)
{
    for(int i=1; i<len; i++)
    {
        _insert_sort(arr,i,arr[i]);
    }
}

void insert_sort(int* arr,size_t len)
{
    for(int i=1; i<len; i++)
    {
        int tmp = arr[i], j = i;
        while(j-1>=0 && arr[j-1]>tmp)
        {
            arr[j] = arr[j-1];
            j--;
```

```
        }
        arr[j] = tmp;
    }
}
```

4、希尔排序

设计该算法的作者叫希尔，所以叫希尔排序，它在插入排序的基础上引入了增量概念(数据在插入时，每次移动的距离)，插入排序默认一次只移动一个位置，当数据量比较大时，移动的速度比较慢，希尔排序先以数量的一半为移动增量，进行插入排序，对数据进行大致排序，然后再减小增量对数据进行微调，进而完成插入排序。

注意：希尔排序适合在数量比较大的时候，向有序的序列中添加新的数据。

```
// 时间复杂度：O(NlogN)，空间复杂度：O(1)，稳定性：不稳定
void shell_sort(int* arr,size_t len)
{
    for(int k=len/2; k>0; k/=2)
    {
        for(int i=k; i<len; i+=k)
        {
            int tmp = arr[i], j = i;
            while(j-k>=0 && arr[j-k]>tmp)
            {
                arr[j] = arr[j-k];
                j-=k;
            }
            arr[j] = tmp;
        }
    }
}
```

5、快速排序

先在待排序的序列中找出一个标杆，然后与剩余的数据进行比较，比标杆小的数据放在标杆的左边，比标杆大的数据放在它右边，这样就做到以标杆为准的大致有序，然后再次同样的方法对标杆左边的数据进行排序、标杆右边的数据进行排序，直到整个序列完全有序。

注意：快速排序之所以叫快速排序，综合各种情况它的表示最好，速度最快，如果对待排序的数据不了解，建议优先选择快速排序。

```
// 时间复杂度：O(NlogN)，空间复杂度：O(logN)，稳定性：不稳定
void _quick_sort(int* arr,int left,int right)
{
    // 备份左右边界
    int l = left, r = right;
    // 把最右边的数据作为标杆，前记录标杆下标
    int pv = arr[left], pi = left;

    while(l<r)
    {
        // 从右向左寻找比标杆小的数据
        while(l<r && arr[r]>=pv) r--;
        // 找到比标杆小的数据
        if(l<r)
        {
            // 把它移动标杆的左边
```

```
            arr[pi] = arr[r];
            // 记录标杆的新位置
            pi = r;
        }
        // 从左向右寻找比标杆大的数据
        while(l<r && arr[l]<=pv) l++;
        // 打到比标杆大的数据
        if(l<r)
        {
            // 把它移动到标杆的右边
            arr[pi] = arr[l];
            // 记录标杆的新位置
            pi = l;
        }
    }
    // 把标杆的值存放到最新位置
    arr[pi] = pv;

    // 快速排序标杆左边的数据
    if(pi-left>1)
        _quick_sort(arr,left,pi-1);
    // 快速排序标杆右边的数据
    if(right-pi>1)
        _quick_sort(arr,pi+1,right);
}
void quick_sort(int* arr,size_t len)
{
    _quick_sort(arr,0,len-1);
}
```

6、堆排序

所谓堆排序就是把待排序的数据当作一个大根堆，然后逐步把堆顶的最大值弹出存储在序列的末尾，也就是借助大根堆这一数据结构完成的排序。

注意：理论上来说堆排序的速度不比快排序慢，但是对无序的序列排序需要先创建堆，时间复杂度是O(N)，然后再逐一出堆完成排序时间复杂度是O(NlogN)，所以对无序的序列排序快速排序比堆的速度要快，所以一般在实际应用中不使用堆排序，只活在教课书中。

```
// 时间复杂度：O(NlogN)，空间复杂度：O(1)，稳定性：不稳定
void _heap_sort(int* arr,int root,size_t len)
{
    while(root*2+1<len)
    {
        // 假定左子树是左右子树中的最大值
        int max = root*2+1;
        // 判断右子树是否大于左子树，如果大于则更新最大值下标
        if(max+1<len && arr[max+1]>arr[max])
            max++;
        // 如果最大子树依然小于根，则结束
        if(arr[max] < arr[root])
            return;
        // 把最大子树与根交换
        swap(arr[max],arr[root]);
        root = max;
    }
}
```

```c
void heap_sort(int* arr,size_t len)
{
    // 创建大根堆
    for(int i=len/2-1; i>=0; i--)
        _heap_sort(arr,i,len);

    for(int i=len-1; i>0; i--)
    {
        // 把堆的根与末尾的数据交换，数量减-1
        swap(arr[0],arr[i]);
        // 调整堆
        _heap_sort(arr,0,i);
    }
    printf("%s:",__func__);
}
```

7、归并排序

把待排序的数据以k=2为单位进行分组，每组分为左右两部分，然后按从小到大的顺序合并到另一块空间，然后k*=2重复该过程，直到k/2>=len则排序完成。

归并排序需要一块额外的空间，用于存储合并的结果，它的时间复杂度与快速、堆相同，但是在排序过程中没有进行数据交换，而直接数据拷贝，因此节约了大量的数据交换的时间，但也耗费了额外的内存，所以它是一各典型的用空间换取时间的排序算法。

注意：如果用户对排序速度有很高的要求，但不在意内存的耗费，适合使用归并排序。

循环语句实现归并排序：

```c
// 时间复杂度：O(NlogN)，空间复杂度：O(N)，稳定性：稳定
void merge_sort(int* arr,size_t len)
{
    // src指向的是待合并的数据，dest用于存储合并后的数据
    int *src = arr, *dest = malloc(sizeof(arr[0])*len);
    for(int k=1; k<len; k*=2)
    {
        // 以k为单位进行合并
        for(int i=0,j=i; i<len; i+=k*2)
        {
            // 计算出左边的起始位置和结束位置
            int l = i, le = i+k < len ? i+k : len;
            // 计算出右边的起始位置和结束位置
            int r = le, re = i+k*2 < len ? i+k*2 : len;
            // 左右两边的数据进行合并，谁小谁存储在dest里面,直到有一方结束
            while(l<le && r<re)
            {
                dest[j++] = src[l]<src[r] ? src[l++] : src[r++];
            }
            // 把左边剩余的数据拷贝到dest里面
            while(l<le)
                dest[j++] = src[l++];
            // 把右边剩余的数据拷贝到dest里面
            while(r<re)
                dest[j++] = src[r++];
        }
        // 合并完成后，dest中存储的就是下一次待合并的数据，所以两个指针交换
        swap(src,dest);
    }
```

```c
    // 执行完swap(src,dest)后，src指向的是排序后的数据，如果src!=arr则说明排序后的数据存储在堆内在中，需要拷贝给arr
    if(src != arr)
    {
        memcpy(arr,src,sizeof(arr[0])*len);
        dest = src;
    }
    // 释放堆内存
    free(dest);
}
```

函数递归实现归并排序：

```c
void __merge_sort(int* src,int* dest,int left,int p,int right)
{
    int l = left, le = p, r = p, re = right, j=left;
    while(l<le && r<re)
    {
        dest[j++] = src[l]<=src[r]?src[l++]:src[r++];
    }
    while(l<le)
        dest[j++] = src[l++];
    while(r<re)
        dest[j++] = src[r++];
    memcpy(src+left,dest+left,sizeof(src[0])*(right-left));
}

void _merge_sort(int* src,int* dest,int left,int right)
{
    int p = (left + right) / 2;
    if(p-left>1)
        _merge_sort(src,dest,left,p);
    if(right-p>1)
        _merge_sort(src,dest,p,right);
    __merge_sort(src,dest,left,p,right);
}

void merge_sort(int* arr,size_t len)
{
    int* dest = malloc(sizeof(arr[0])*len);
    _merge_sort(arr,dest,0,len);
    free(dest);
}
```

8、计数排序

首先定义一个计算数据出现的次数的数组cnts，并所有成员初始化0，然后使用数据的值作为数组的下标，然后统计每个数据出现的次数。

以i=[0,max]遍历cnts数组，当cnst[i]>0说明i出现过，然后把i往待排序的数组中存储cnts[i]个，然后排序完成。

注意：计算排序的局限比较大，它只能对整型数据进行排序，无法对浮点型、字符串型数据进行排序，待排序的数据重复性越高，差值越小，速度就越快，反之虽然也可以排，但得不尝失。

知道待排序的数据范围：

```
// 时间复杂度：O(N+K)，空间复杂度：O(K)，稳定性：稳定
void count_sort(int* arr,size_t len)
{
    int cnts[100] = {};
    for(int i=0; i<len; i++)
    {
        cnts[arr[i]]++;
    }

    int k = 0;
    for(int i=0; i<100; i++)
    {
        for(int j=0; j<cnts[i]; j++)
        {
            arr[k++] = i;
        }
    }
}
```

不知道待排序的数据范围：

```
void count_sort(int* arr,size_t len)
{
    int max = arr[0], min = arr[0];
    for(int i=1; i<len; i++)
    {
        if(arr[i] > max)
            max = arr[i];
        if(arr[i] < min)
            min = arr[i];
    }

    int* cnts = calloc(max-min+1,sizeof(arr[0]));
    for(int i=0; i<len; i++)
    {
        cnts[arr[i]-min]++;
    }

    for(int i=min,k=0; i<=max; i++)
    {
        for(int j=0; j<cnts[i-min]; j++)
        {
            arr[k] = i;
        }
    }
    free(cnts);
}
```

9、基数排序

先根据数据个位的大小对数据进行排序，然后再对排序结果的十位进行排序，然后百位、千位...，直到排序完成。

使用这种方式排序的优点是不需要对待排序的数据进行比较、交换，所以它的排序速度要比普通排序快的多，但局限性很大，只能对整型数据排序，还需要额外的内存空间。

注意：当数据的位数不多，并且差别不大，的整形数据适合使用基数排序。

```
// 时间复杂：O(n+k) 空间复杂度：O(n+k) 稳定性：稳定
void radix_sort(int* arr,size_t len)
{
    int (*radix)[len+1] = calloc(10,sizeof(arr[0])*(len+1));

    int exp = 1;
    while(exp < 1000000000)
    {
        for(int i=0; i<len; i++)
        {
            int row = arr[i]%(exp*10)/exp;
            radix[row][++radix[row][0]] = arr[i];
        }
        exp *= 10;

        int index = 0, cnt = 0;
        for(int i=0; i<10; i++)
        {
            for(int j=1; j<=radix[i][0]; j++)
            {
                arr[index++] = radix[i][j];
                if(radix[i][j] <exp)
                    cnt++;
            }
            radix[i][0] = 0;
        }
        if(cnt >= (len-1))
            break;
    }
    free(radix);
}
```

10、桶排序

桶排序就是把待排序的数据，根据值划分不同的范围存储到不同的"桶"中，然后再使用其它排序算法对桶中的数据进行排序，最终再合并桶中的数据达到排序的目的。

之所以这样，是因为待排序的数据比较大时，会影响排序算法的性能，桶排序就是通过数据进行分类，降低数据的规模，从而提高排序算法的性能。

注意：该算法代码简单，但实现难度很大，需要对数据有足够的了解，每个桶中数量要够均匀，才能达到预期效果。

```
// 时间复杂：O(n+k) 空间复杂度：O(n+k) 稳定性：未知
void bucket_sort(int* arr,size_t len)
{
    int bucket[3][len],i1=0,i2=0,i3=0;
    for(int i=0; i<len; i++)
    {
        if(arr[i] < 100)
            bucket[0][i1++] = arr[i];
        else if(arr[i] < 1000)
            bucket[1][i2++] = arr[i];
```

```
        else
            bucket[2][i3++] = arr[i];

    }
    quick_sort(bucket[0],i1);
    quick_sort(bucket[1],i2);
    quick_sort(bucket[2],i3);

    memcpy(arr,bucket[0],sizeof(arr[0])*i1);
    memcpy(arr+i1,bucket[1],sizeof(arr[0])*i2);
    memcpy(arr+i1+i2,bucket[2],sizeof(arr[0])*i3);
}
```